High-Level Design (HLD) Interview Guide

System design problems with full decision reasoning (API, DB, LB, protocol, caching, failure, multi-geo)

Target level: Mid-level software engineering interviews at companies such as Booking.com, Airbnb, Microsoft, Agoda, UiPath, and organizations of similar range.

How this guide is structured

An HLD round is not about drawing the "right" diagram — it's about **defending every choice**. When you put a load balancer on the board, the interviewer asks "*L4 or L7? Why?*" When you pick a database, "*SQL or NoSQL? Why not the other?*" This guide drills that reasoning into every problem via a **fixed 15-point checklist**, so you never freeze on a follow-up.

Every problem answers, in order:

```
 1. Requirements        functional + non-functional (scale, latency,
                        consistency vs availability)
 2. Estimation          QPS, storage, bandwidth — with the math
 3. API style           REST vs gRPC vs GraphQL — and WHY
 4. Idempotency         do we need idempotency keys? why / why not
 5. Network protocol     HTTP/1.1, HTTP/2, WebSocket, gRPC/HTTP2 — and why
 6. Database             SQL vs NoSQL — and WHY, with the access pattern
 7. Data model           detailed schema / key design
 8. Load balancer        L4 vs L7 vs API Gateway — which benefits we use
 9. Caching + data flow   how data moves between DB and cache
10. Concurrency          where shared state races, and how we guard it
11. Replication          DB replicas needed? leader-follower? quorum?
12. Multi-geo            multi-region strategy, data locality, latency
13. Failure handling     what breaks, and how we degrade gracefully
14. Bottlenecks & scaling where it breaks first and the fix
15. Tradeoffs + follow-ups the crisp senior-signal summary
```

Decision frameworks (memorize these — they answer 80% of follow-ups)

API style: REST vs gRPC vs GraphQL

```
REST     - resource CRUD over HTTP/JSON; universal, cacheable, simple.
           DEFAULT for public/external APIs and simple services.
gRPC     - binary Protobuf over HTTP/2; low latency, streaming,
           strongly typed. BEST for internal service-to-service,
           high-throughput, polyglot microservices.
GraphQL  - client picks exactly the fields it needs; one round trip
           for nested data. BEST for mobile/rich UIs with many
           resource shapes and over-fetching pain (e.g., feeds).
```

How to answer: "External clients → REST for ubiquity and caching. Internal hot paths → gRPC for latency and streaming. Client-driven aggregation over many entities → GraphQL."

Idempotency: do we need keys?

```
NEED idempotency keys when a retry could DUPLICATE a side effect:
  - payments, bookings, order creation, "create" POSTs, money moves.
  Client sends Idempotency-Key: <uuid>; server dedupes on it.
DON'T strictly need them when the operation is naturally idempotent:
  - GET (safe), PUT to an absolute value, DELETE by id.
```

Why: networks retry. Without a key, "charge $50" retried = charged twice. The key lets the server recognize "I already processed this request" and return the first result.

Database: SQL vs NoSQL

```
SQL (Postgres/MySQL)   - strong consistency, ACID transactions, joins,
                         flexible queries. PICK when relationships and
                         multi-row transactions matter (bookings, payments,
                         inventory). Scales via read replicas + sharding.
NoSQL - KV (Redis/Dynamo)   - O(1) by key, massive scale, low latency.
      - Document (Mongo)     - flexible schema, nested docs.
      - Wide-col (Cassandra) - write-heavy, time-series, tunable
                               consistency, multi-DC replication.
      PICK when the access pattern is known + key-based, write volume is
      huge, or you need horizontal scale over strict consistency.
```

How to answer: state the **dominant access pattern first**, then the DB. "Reads are by primary key at very high QPS with no joins → Dynamo/Cassandra. I need multi-item transactions and consistency → Postgres."

Load balancer: L4 vs L7 vs API Gateway

```
L4 (transport) - routes by IP/port, no payload inspection. Fastest,
                 cheapest. Use for raw TCP/UDP, or in front of L7.
L7 (application)- reads HTTP: path/header/cookie routing, TLS
                 termination, sticky sessions, content-based routing.
API Gateway    - L7 PLUS auth, rate limiting, request validation,
                 routing to microservices, aggregation, observability.
                 Use as the single front door for a microservice system.
```

How to answer: "Global entry → L7 for path-based routing + TLS termination. In a microservice system I front it with an API Gateway for auth, rate limiting, and routing. L4 only where I need raw speed or non-HTTP traffic."

Network protocol

```
HTTP/1.1  - simple, ubiquitous; head-of-line blocking per connection.
HTTP/2    - multiplexed streams, header compression; gRPC rides on it.
WebSocket - full-duplex, persistent; PICK for chat, live updates,
            presence, real-time notifications.
UDP/QUIC  - low latency, lossy-tolerant; media, gaming, HTTP/3.
```

Caching data flow (cache-aside, the default)

```
READ:  app -> check cache
         hit  -> return
         miss -> read DB -> write to cache (with TTL) -> return
WRITE: app -> write DB -> INVALIDATE (or update) cache key
  Why invalidate over update? avoids stale writes racing; next read
  repopulates. Write-through/write-back are alternatives (note tradeoffs).
```

Replication & multi-geo (the quick rules)

```
Read replicas   - when reads >> writes. Async replication -> replica lag
                  -> reads may be slightly stale (eventual consistency).
Leader-follower - one writer, many readers; failover promotes a follower.
Quorum (W+R>N)  - Dynamo/Cassandra tunable consistency.
Multi-geo       - route users to nearest region (GeoDNS/anycast).
                  - Active-passive: one write region + DR.
                  - Active-active: writes in multiple regions ->
                    conflict resolution (last-write-wins / CRDTs).
                  - Keep data local to region for latency + compliance.
```

Numbers worth knowing (order of magnitude)

Operation	~Latency
L1 / memory reference	~1–100 ns
Read 1 MB from RAM	~10 µs
SSD random read	~100 µs
Round trip within a datacenter	~0.5 ms
Read 1 MB from SSD	~1 ms
Round trip across continents	~100–150 ms

Estimation shortcuts: 1 day ≈ 86,400 s ≈ 10^5 s. So 1M requests/day ≈ ~12 QPS average; plan for peak = 2–3× average. 1 KB × 1M = 1 GB.

The 12 problems

#	Problem	Headline challenge
1	URL Shortener	Key generation, read-heavy caching
2	Pastebin	Blob storage + metadata, expiry
3	Distributed Rate Limiter	Atomic counters across nodes
4	Key-Value Store / Cache	Consistent hashing, replication
5	Unique ID Generator	Ordered, unique, no coordination (Snowflake)

#	Problem	Headline challenge
6	Hotel / Flight Booking	Double-booking prevention, transactions
7	Airbnb Listing + Search + Booking	Availability, geo-search
8	News Feed	Fan-out on write vs read
9	Chat / Messaging	WebSockets, delivery, presence
10	Notification System	Multi-channel, queues, retries
11	Ride-Hailing	Geo-indexing, matching
12	Nearby Places / Proximity	Geohash / quadtree search

1. URL Shortener (TinyURL / bit.ly)

The problem in one line. Take a long URL and return a short alias; when someone hits the short alias, redirect them to the original — at massive read scale, fast.

---

1. Requirements

Functional - `shorten(longUrl)` → returns a short URL (e.g., `sho.rt/aB3xK9`). - `GET /aB3xK9` → HTTP 301/302 redirect to the long URL. - Optional: custom alias, expiry date, analytics (click counts).

Non-functional - **Read-heavy**: redirects vastly outnumber creations (~100:1). - **Low latency** on redirect (< 50 ms) — it's in the user's critical path. - **High availability** — a dead shortener breaks every link that uses it. - Short codes are **not guessable/sequential** (optional, for privacy).

2. Estimation

```
Assume 100M new URLs/month.
  writes/sec = 100M / (30 * 86400) ~= 40 writes/sec
  reads/sec  = 100:1 ratio         ~= 4,000 reads/sec (peak ~10k)
Storage: 100M/month * 12 * 5 yrs = 6B records.
  per record ~500 bytes (short code + long URL + metadata)
  6B * 500B = ~3 TB over 5 years.  (fits sharded SQL or a KV store)
Short code length: base62 [a-zA-Z0-9].
  62^6 = 56 billion combos -> 6 chars is plenty for years.
```

3. API style — REST vs gRPC vs GraphQL

Choice: REST (public), optionally gRPC for the internal write service.

```
POST /api/v1/urls        { longUrl, customAlias?, ttl? } -> { shortUrl }
GET  /{shortCode}        -> 301/302 Location: <longUrl>
```

Why REST: the redirect endpoint must be a plain HTTP GET that any browser can hit — that *is* REST. Creation is a simple resource POST; no need for GraphQL's field selection (one tiny response shape) or gRPC's streaming. **Why not GraphQL:** there's no nested/over-fetch problem to solve. **Why gRPC internally:** if a separate write service does ID allocation, gRPC gives lower latency and typed contracts between services.

4. Idempotency — needed?

Mostly no, with a nuance. - The redirect `GET` is **naturally idempotent** (safe read) — no key needed. - For `shorten`: if the same `longUrl` from the same user should always map to the **same** short code, make creation deterministic (hash the URL) — then retries are idempotent by design. If every call must

mint a **new** code, then a client `Idempotency-Key` prevents a network retry from creating two codes for one intended action. **Say this tradeoff aloud.**

5. Network protocol

HTTP/1.1 or HTTP/2 over TLS. Redirects are classic request/response — no need for WebSocket (no server push) or persistent duplex. HTTP/2 helps if a page triggers many lookups. TLS everywhere.

6. Database — SQL vs NoSQL

Choice: a KV store (DynamoDB / Cassandra) for the mapping; optionally SQL if you need rich analytics queries.

Why: the dominant access pattern is a **single-key lookup** — `shortCode → longUrl` — at thousands of QPS with **no joins and no multi-row transactions**. That's the textbook KV case: O(1) by key, horizontal scale, low latency. **Why not SQL as primary:** we don't need joins or ACID transactions on the hot path; a relational DB's strengths are wasted and it's harder to scale writes horizontally. (If click analytics need ad-hoc queries, stream events to a separate OLAP store — don't burden the redirect path.)

7. Data model (detailed)

```
Table: urls  (KV / wide-column)
-----------------------------------------------------
 short_code   (STRING)  PARTITION KEY   e.g. "aB3xK9"
 long_url     (STRING)                  the destination
 created_at   (TIMESTAMP)
 expires_at   (TIMESTAMP, nullable)     for TTL cleanup
 owner_id     (STRING, nullable)        who created it
 hit_count    (COUNTER, optional)       or track in analytics store
-----------------------------------------------------
 Lookup: GET by short_code  -> O(1) partition read.

Optional Table: url_by_hash (for dedup of identical longUrls)
 url_hash (STRING) PK  -> short_code    (so same URL reuses a code)
```

Short code generation — two approaches:

```
A) Hash + encode:  code = base62( hash(longUrl + salt) )[:7]
     + deterministic (same URL -> same code, natural dedup)
     - collisions possible -> check-and-retry with longer/altered code
B) Counter + base62:  a global unique ID -> base62 encode it
     + no collisions ever (IDs are unique)
     - needs a distributed ID generator (see problem #5); sequential
       unless you scramble the bits
```

8. Load balancer — L4 vs L7 vs API Gateway

```
  Client
    |
[ GeoDNS ]  -> nearest region
    |
[ L7 LB / API Gateway ]  <- TLS termination, path routing,
    |                       rate limiting, auth on the write path
    +--> Redirect Service (read, stateless, autoscaled)
    +--> Shorten Service  (write)
```

Choice: L7 / API Gateway at the edge. We route `POST /api/*` (write) and `GET /{code}` (read) to **different autoscaling groups** — that's **path-based routing**, an L7 feature. The gateway also gives us **TLS termination, rate limiting** (stop abuse of the create API), and **auth** for owned links. **Why not pure L4:** L4 can't route by path or terminate TLS. We *may* put L4 in front of L7 for raw connection distribution at extreme scale, but L7 is where the useful decisions happen.

9. Caching + data flow (DB ↔ cache)

Redirects are read-heavy and the mapping is immutable once created → **cache aggressively.**

```
REDIRECT read path (cache-aside):
  GET /aB3xK9
    -> check Redis for "aB3xK9"
         HIT  -> return longUrl (99%+ of traffic)
         MISS -> read KV store -> put in Redis (long TTL) -> return
WRITE path:
  POST -> write to KV store -> (optionally pre-warm Redis)
  Mappings rarely change, so invalidation is trivial; on expiry/delete
  we remove the key from both stores.
```

A **CDN / edge cache** can even serve hot redirects without hitting origin. Cache hit ratio is extremely high because a small set of links gets most clicks (power law).

10. Concurrency

- **Code-generation race:** two writers could generate the same code. Guard with a **conditional write** ("insert if `short_code` not exists" — DynamoDB `attribute_not_exists` / Cassandra LWT). On conflict, regenerate.
- **Counter approach** avoids the race entirely (IDs are globally unique by construction).
- `hit_count` under concurrent clicks → use an **atomic counter** or, better, aggregate asynchronously from a stream (don't do a synchronous DB write per redirect).

11. Replication — needed?

Yes. Reads dominate, so: - **Multiple read replicas** (or a naturally replicated store like Dynamo/Cassandra with `RF=3`) to serve high read QPS and survive node loss. - Mappings are immutable → **replica lag is harmless** (a slightly stale replica still has the correct, unchanging mapping). This is why eventual consistency is perfectly fine here.

12. Multi-geo

```
GeoDNS routes users to the nearest region.
Mapping data is immutable + read-mostly -> replicate the whole dataset
to every region (active-active reads). Writes can go to a home region
and replicate out; a few seconds of propagation lag is acceptable
because a brand-new link isn't expected to resolve globally instantly.
```

Benefit: redirect latency stays low worldwide; compliance is easy since the data isn't user-PII-heavy.

13. Failure handling

```
Cache (Redis) down    -> fall back to KV store reads (slower, still works).
A KV node down        -> RF=3 + quorum reads keep serving.
Region down           -> GeoDNS reroutes to next region (data is
                         replicated everywhere).
Write service down    -> redirects (reads) UNAFFECTED; only new-link
                         creation pauses -> degrade gracefully, queue writes.
Bad/expired code      -> return 404 with a friendly page, never 500.
```

Design principle: **the read path must survive write-path failure**, because a broken redirect breaks the whole product.

14. Bottlenecks & scaling

- **Redirect QPS** → stateless read services autoscale horizontally behind the LB; Redis + CDN absorb most traffic.
- **Storage growth** → the KV store shards by `short_code` (its partition key) automatically.
- **Hot keys** (a viral link) → CDN/edge cache and Redis handle it; replicas spread load.
- **Analytics writes** → never on the hot path; emit events to Kafka → aggregate offline.

15. Tradeoffs & what to say

> "It's a read-heavy, single-key lookup, so I use a KV store with heavy cache-aside caching and a CDN — reads scale horizontally and tolerate stale replicas because mappings are immutable. Short codes come from a distributed counter (no collisions) or a hash (natural dedup). REST because the redirect is just an HTTP GET; L7/API-gateway for path routing, TLS, and rate-limiting the create API. Replication with RF=3 and multi-region active-active reads keeps it fast and available; the read path is isolated so it survives write-path failure."

Follow-ups an interviewer may probe - How do you guarantee uniqueness without a single point of contention? (Snowflake-style ID, or per-node ID ranges — see problem #5.) - 301 vs 302? (301 is cached by browsers → fewer hits but no analytics; 302 keeps control.) - How to expire billions of links? (TTL attribute + background cleanup / store TTL.) - Preventing abuse (malicious URLs)? (Safe-browsing check on create, rate limits.)

---

2. Pastebin

The problem in one line. Store a chunk of text/code and hand back a short key; anyone with the key can fetch the paste back — some pastes expire, some self-destruct after one read.

---

1. Requirements

Functional - `createPaste(content, ttl?, visibility?)` → returns a short key (e.g., `pb.io/x7Qa2`). - `GET /{key}` → returns the paste content (raw text or rendered). - **Expiry**: time-based TTL (e.g., "delete after 1 day") and **burn-after-read** (delete the instant it's viewed once). - **Visibility**: public (listable/indexable) vs unlisted (only reachable via the key). - Content size: plain text/code, up to a few **MB** per paste (not GB — that's a file-storage problem, not a pastebin problem).

Non-functional - **Read-heavy-ish**, but not as skewed as a URL shortener — pastes are viewed a handful of times, not millions (no long-tail viral redirect pattern to lean on as hard). - **Payload size varies a lot** (a one-line snippet vs. a 5 MB log dump) — the system must not choke the metadata store with big blobs. - Durability: a paste shouldn't vanish before its TTL; but it's not "financial data" — best effort with reasonable durability (object storage's 11-nines is more than enough). - Low latency on read (< 200 ms) is nice-to-have, not as brutally critical as a redirect.

2. Estimation

```
Assume 5M new pastes/day, avg size 50 KB (mix of tiny snippets and
multi-MB logs), average views per paste ~5.

  writes/sec = 5M / 86400        ~= 58 writes/sec  (peak ~200)
  reads/sec  = 5x writes         ~= 290 reads/sec  (peak ~1,000)

Storage (blob):
  5M/day * 50KB = 250 GB/day -> * 365 * 2 yrs (many expire sooner)
  ~= tens of TB steady-state after expiry/GC nets most of it out.
  -> object storage (S3-like), NOT a relational DB's row storage.

Storage (metadata): ~200 bytes/row (key, size, ttl, owner, flags)
  5M/day * 200B * 365 * 2yrs ~= ~700M rows / ~150 GB -> trivial for
  a KV store or sharded SQL, even before expiry cleanup shrinks it.

Key length: base62, 7 chars -> 62^7 ~= 3.5 trillion combos. Plenty.
```

The **shape** of the estimate is the point: metadata is small and steady; blob storage is the big, bursty number — which is exactly why they get split onto different systems (section 6).

3. API style — REST vs gRPC vs GraphQL

Choice: REST.

```
POST /api/v1/pastes   { content, ttlSeconds?, burnAfterRead?, visibility? }
                       -> { key, url, expiresAt? }
GET  /api/v1/pastes/{key}   -> { content, createdAt, expiresAt? }
DELETE /api/v1/pastes/{key} -> (owner-only, manual delete)
```

Why REST: this is classic CRUD over a single resource (`paste`) — POST to create, GET to fetch, DELETE to remove. No nested/relational data to selectively fetch (ruling out GraphQL's main advantage), and no bidirectional streaming need (ruling out gRPC for the public API). **Why not GraphQL:** a paste is a flat object; there's no over-fetching problem to solve. **Why not gRPC (public):** browsers need to `curl`/link directly to a raw-text URL; gRPC isn't naturally browser-fetchable. gRPC could still make sense **internally** between the API service and a storage-write microservice, but the public surface is plain REST/HTTP.

4. Idempotency — needed?

Yes, on create — more than the URL shortener, because payloads are bigger and retries are more likely (slow uploads time out and get retried by the client). - `GET` and `DELETE` are naturally idempotent (safe/repeatable). - `POST /pastes` is **not** naturally idempotent: a network blip causing a client retry could create two identical pastes with two different keys (wasted storage) — or worse, if the client *thinks* the first attempt failed but it actually succeeded, the user now has two live links to "the same" secret paste. - Fix: accept a client-generated **`Idempotency-Key` header** (or dedupe by `hash(content) + owner` within a short window). Server caches "key → result" for e.g. 24h; a retry with the same idempotency key returns the original `{key, url}` instead of creating a duplicate.

5. Network protocol

HTTP/1.1 or HTTP/2 over TLS, plain request/response. No WebSocket/streaming needed for create-then-fetch. HTTP/2 helps if a page loads several pastes' worth of assets at once. For large pastes, support **chunked upload / `Content-Length`** and gzip so a 5 MB paste doesn't block on one huge synchronous write. TLS everywhere (pastes often contain secrets/code).

6. Database — SQL vs NoSQL

Choice: split the data — metadata in a KV/SQL store, the actual blob in object storage (S3-like), with a CDN in front of blob reads.

Why the split (the key insight of this problem): - The **access pattern** for metadata (key → owner, ttl, flags, size, blob pointer) is a tiny, single-key lookup — the same KV-friendly shape as the URL shortener. - The **blob** (the actual paste text, up to several MB) is large, immutable once written, and read by content rather than queried. Databases are optimized for small rows and fast transactional access to *structured* fields — stuffing multi-MB blobs into DB rows causes: - **Row/page bloat** — a few large rows blow up buffer-pool/cache efficiency for every other (tiny) row that happens to share pages/segments with it. - **Cost** — DB storage (esp. provisioned SSD-backed) is far more expensive per GB than object storage. - **Backup/replication overhead** — every DB snapshot/replica now has to move all that blob weight, even though it never changes after creation. - **No CDN-ability** — a DB row isn't something a CDN or browser cache can serve directly; an object-storage URL (or a signed URL) is. - **Why KV/document over SQL for**

the metadata itself: the lookup is single-key with no joins — same reasoning as URL shortener. SQL is fine too at this metadata volume; it isn't the bottleneck either way. **The blob's home is the real decision.**

```
Client -> API -> [ metadata: KV/SQL ]   (key, ttl, owner, blob pointer)
               -> [ blob store: S3 ]    (the actual text, immutable)
               -> [ CDN ]               (caches blob reads at the edge)
```

7. Data model (detailed)

```
Table: paste_metadata   (KV or SQL row)
-------------------------------------------------------------------
 paste_key        (STRING)   PARTITION KEY   e.g. "x7Qa2Bd"
 owner_id         (STRING, nullable)         anonymous allowed
 visibility       (ENUM: public|unlisted)
 size_bytes       (INT)                      for quota/limits
 content_type     (STRING)                   e.g. "text/plain", "text/x-python"
 blob_uri         (STRING)                   e.g. "s3://pastes/2026/07/x7Qa2Bd"
 created_at       (TIMESTAMP)
 expires_at       (TIMESTAMP, nullable)       TTL cleanup target
 burn_after_read  (BOOL)                      delete-on-first-view flag
 read_count       (INT, optional)             for analytics / burn logic
-------------------------------------------------------------------
 Lookup: GET by paste_key -> O(1) partition read -> then fetch blob_uri.

Object store layout (S3-like):
 bucket/pastes/{yyyy}/{mm}/{paste_key}     <- raw content, one object
 (partitioning by date makes lifecycle-based expiry/GC rules trivial)
```

Keeping `content_type` / `size_bytes` in metadata lets the API decide render-as-text vs. download, and enforce size limits, **without ever touching the blob store** for a HEAD-like check.

8. Load balancer — L4 vs L7 vs API Gateway

```
  Client
    |
[ L7 LB / API Gateway ]  <- TLS termination, path routing,
    |                       rate limiting, request size limits,
    |                       auth for owner-only delete
    +--> Create Service (write: metadata + blob)
    +--> Read Service    (read: metadata lookup + blob fetch)
    +--> (static/raw fetch of blobs may bypass app servers via
          CDN/S3 pre-signed URLs entirely)
```

Choice: L7 / API Gateway. We need **path-based routing** (`POST /pastes` vs. `GET /pastes/{key}` to different pools), **request-size limits** at the edge (reject oversized uploads before they hit app servers — an L7-only capability since it inspects the body/headers), and **rate limiting** to stop abuse (spam pastes, scraping). **Why not pure L4:** L4 only sees IP/port, can't enforce a max-body-size or route by path/method. L4 could still sit in front of the L7 tier purely for TCP-level load spreading, but the interesting decisions (routing, limits, TLS) all happen at L7.

9. Caching + data flow (DB ↔ cache)

```
CREATE flow:
  POST /pastes
    -> write blob to S3 first             (blob_uri now valid)
    -> write metadata row (blob_uri, ttl) second
    -> return {key, url}
  (order matters -- see section 13: never point metadata at a blob
   that doesn't exist yet)

READ flow (cache-aside):
  GET /pastes/{key}
    -> check Redis for metadata["key"]
         HIT  -> use cached blob_uri
         MISS -> read metadata store -> cache it (TTL-bounded by the
                 paste's own expires_at, never longer)
    -> fetch blob from S3, but prefer CDN/edge cache in front of S3
         CDN HIT  -> return content directly, app server untouched
         CDN MISS -> fetch from S3, populate CDN, return
```

Pastes are **immutable once created** (no edits), so cache invalidation is simple: entries just need to expire no later than the paste's own TTL, and burn-after-read pastes must **bypass/invalidate cache on the read that consumes them** (see concurrency below) so a second request never serves cached content for a paste that should already be gone.

10. Concurrency

- **Burn-after-read race (the sharp edge of this problem):** two concurrent `GET`s for the same burn-after-read key must not both succeed in reading the content. The read-then-delete must be a single **atomic conditional operation**: `SQL: DELETE FROM paste_metadata WHERE paste_key = ? AND burn_after_read = true RETURNING blob_uri; -- exactly one caller gets a row back; the loser gets "not found" KV: conditional delete-and-return (e.g., DynamoDB DeleteItem with ConditionExpression, returning old values)` Whoever wins the atomic delete serves the content (fetches blob, returns it, then deletes the blob async); the loser gets a 404 exactly as if the paste never existed.
- **Create-time key collision:** same guard as URL shortener — conditional insert (`attribute_not_exists` / unique constraint) with regenerate-on-conflict.
- **`read_count` increments** under concurrent reads → atomic counter, or better, don't do a synchronous write per read at all — emit a read event to a stream and aggregate async.

11. Replication — needed?

Yes, for both tiers, for different reasons. - **Metadata store:** replicate (leader-follower or quorum-based, RF=3) for read scaling and node-failure survival, same as URL shortener — mappings don't change after creation, so replica lag is harmless for content correctness. The one place staleness *does* matter is burn-after-read: the atomic delete-on-read must go through a strongly consistent path (leader/quorum write), not an eventually-consistent replica, or two replicas could both think the paste is unread. - **Blob store:** object stores like S3 already replicate internally (multi-AZ) for durability; no extra work needed beyond choosing a durable storage class.

12. Multi-geo

```
Metadata: replicate across regions (read-mostly, immutable once
written) -> active-active reads, home-region writes propagate out.
Blobs: use the object store's built-in cross-region replication,
OR simply serve all regions through a CDN with edge PoPs -- the
origin (S3) can stay in one region since blob reads are cacheable
and immutable, so edge caching solves geo-latency without full
blob replication in most cases.
```

Burn-after-read is the one flow that needs care cross-region: route the atomic delete-on-read to the metadata's home/leader region (or a globally consistent store) so "burn" is enforced exactly once even if requests land in different regions.

13. Failure handling

```
Blob write succeeds, metadata write fails
  -> orphaned blob in S3, no paste ever "exists" to users (metadata
     is the source of truth) -> harmless; a GC sweep deletes blobs
     with no matching metadata row after some grace period.
Metadata write succeeds, blob write failed
  -> MUST NOT HAPPEN -> this is why we write blob FIRST, metadata
     SECOND: metadata should never point at a blob that isn't there.
Object store (S3) down
  -> creates fail fast with a clear error (don't silently accept
     metadata for a paste with no content); reads of already-cached
     (CDN/Redis) content can still succeed for a while.
Metadata store node down
  -> replicas/quorum keep serving; burn-after-read may briefly
     refuse writes rather than risk a double-read (fail closed).
Cache/CDN down
  -> fall back to direct blob store reads (slower, still correct).
Expired/missing key requested
  -> 404, never 500; don't leak whether a key ever existed if
     visibility is unlisted (avoid enumeration hints).
```

Design principle: **order writes so the metadata store is only ever a pointer to blobs that truly exist** — inconsistency should always fail in the "safe" direction (orphaned blob, never a dangling metadata pointer).

14. Bottlenecks & scaling

- **Large uploads** → stream/chunk the write to the blob store rather than buffering the whole payload in app-server memory; enforce a max size at the LB/gateway.
- **Blob store read load** → CDN absorbs the vast majority; origin (S3) only sees cache misses and cold/rare pastes.
- **Metadata store growth** → TTL-based expiry plus a background sweeper deletes rows (and triggers blob deletion) past `expires_at`, keeping the working set small; shard by `paste_key`.
- **Burn-after-read contention** → rare in practice (one key, viewed once) but must stay on a strongly consistent path even as the system scales horizontally — isolate it from the eventually-consistent bulk-read path so it doesn't force the whole metadata tier to sacrifice read scalability.

- **Hot pastes** (a snippet goes viral, e.g. shared in a popular thread) → CDN caching handles it exactly like a hot URL-shortener redirect.

15. Tradeoffs & what to say

> "The core insight is separating small, structured metadata from a large, immutable blob: metadata lives in a KV/SQL store for fast single-key lookups, the actual paste text lives in object storage behind a CDN, because databases are the wrong tool for multi-MB rows — cost, row bloat, and no CDN-ability. Writes go blob-first, metadata-second, so metadata never points at a blob that doesn't exist; failures then always fail safe (an orphaned blob we GC, never a dangling pointer). Burn-after-read is the one tricky concurrency spot — it needs an atomic conditional delete-on-read so two simultaneous viewers can't both succeed. REST for a plain CRUD resource, L7/API-gateway for path routing and size/rate limits, replication for both tiers (metadata for read scaling, object storage's built-in multi-AZ for blob durability), and CDN/edge caching to keep reads fast globally without replicating every blob everywhere."

Follow-ups an interviewer may probe - How do you enforce max paste size fairly? (Limit at the gateway/LB before it reaches app servers or blob store.) - How would you support paste *editing* (revisions)? (Breaks the "immutable blob" assumption — either version each edit as a new blob + a version list in metadata, or explicitly say pastes are append-only/immutable by design.) - How do you avoid abuse (malware/spam pastes)? (Rate limit creates per IP/account, async content scanning, CAPTCHA for anonymous creation.) - How do you clean up expired pastes at scale? (TTL attribute + background sweep job, or native TTL support in the KV store; pair with an S3 lifecycle rule keyed off the same age/prefix.)

---

3. Distributed Rate Limiter

The problem in one line. Decide, in microseconds and across many gateway nodes, whether `clientId`'s next request is allowed or must be rejected — without letting the nodes disagree with each other.

---

1. Requirements

Functional - `allow(clientId, endpoint)` → `allow` or `deny`, called on (almost) every request. - Configurable limits: e.g. 100 requests/min per user, 1000/min per API key, different caps per endpoint (`/login` stricter than `/search`). - Return enough info for a `429` response: `Retry-After`, remaining quota. - Optional: burst allowance (short spikes above the steady rate).

Non-functional - **Very low added latency** — this check sits in front of every request; it must add microseconds, not milliseconds. - **Correctness under concurrency** — with N gateway nodes all checking the same client's counter, the limit must hold *globally*, not per-node. - **High availability** — the limiter must never become the reason the whole API is down. - **Horizontally scalable** — must handle the same fan-in as the API layer it protects.

2. Estimation

```
Assume 500k req/sec across the fleet, 200 gateway nodes.
  -> each node does ~2,500 checks/sec locally.
Each check = 1 Redis round trip (if centralized):
  Redis single instance handles ~100k ops/sec easily -> 500k ops/sec
  needs a small Redis cluster (5-10 shards) or local caching to cut
  the fan-in.
Counter storage: 50M distinct clients, one counter per (client, window).
  50M * ~100 bytes (key + int + TTL bookkeeping) = ~5 GB in Redis.
  -> fits comfortably in memory; no disk needed.
Latency budget: added hop to Redis in the same AZ ~= 0.5-1ms.
  That's the whole design tension: 0.5-1ms * every request adds up
  at 500k/sec, so local caching / batching matters (see #3, #9).
```

3. API style — REST vs gRPC vs GraphQL

Choice: internal gRPC (or an in-process library call), NOT a public REST API.

```
Gateway node                 Rate Limiter (library or sidecar)
    |  check(clientId, rule)        |
    |------------------------------>|  (in-process call, or gRPC
    |<------------------------------|   to a local sidecar)
    |  {allow: bool, retryAfterMs}  |
```

Why gRPC/in-process, not REST: this call happens on the hot path of *every single request* the gateway serves — REST's JSON + HTTP framing overhead is wasted cost here. A tight binary protocol (gRPC) or,

even better, an **in-process library** that talks to Redis directly, keeps the added latency minimal. **Why not GraphQL:** there is exactly one shape of request/response; no client-driven field selection to justify it. If the rate limiter is its own microservice (shared across many gateways), gRPC to a **local sidecar** (not a cross-AZ hop) is the practical middle ground — see the "remote vs local" tradeoff in #9.

4. Idempotency — needed?

No — and it must NOT be idempotent in the usual sense. - Idempotency is about *retries producing the same effect*. A rate-limit check is the opposite: it's meant to have a **side effect every time** (consume one unit of quota). Making `allow()` idempotent (e.g., keying on a request ID so repeats don't double-count) would let a client dodge the limiter by resending the same idempotency key forever. - The one place idempotency *does* matter: if the caller retries a check after a **timeout** where it's unsure whether the previous attempt was counted, that ambiguity is inherent to the problem, not something to paper over — see fail-open/fail-closed in #13.

5. Network protocol

Internal gRPC (HTTP/2) between gateway and a limiter sidecar/service; the Redis protocol (RESP) between the limiter and its counter store. No WebSocket/long-lived streaming needed — each check is a single, fast request/response. Keep connections **persistent and pooled** (gRPC channel reuse, Redis connection pool) so we're not paying TCP/TLS handshake cost per check — that overhead alone would blow the latency budget from #2.

6. Database — SQL vs NoSQL

Choice: Redis (in-memory key-value store). Not SQL, not even a general NoSQL document store — this specifically wants an in-memory KV/counter store.

Why: - **Speed**: counters live entirely in RAM; a single `INCR` is sub-millisecond. A disk-backed DB (SQL or otherwise) adds an order of magnitude of latency we don't have budget for. - **Atomic primitives built in**: `INCR`, `EXPIRE`, and **Lua scripting** (`EVAL`) let us do "read counter, compare to limit, increment, set TTL" as **one atomic server-side operation** — exactly what a check-and-increment rate limiter needs (see #10). - **Native TTL**: a counter for "requests in this 1-minute window" should vanish after a minute. `EXPIRE` / `SET ... EX` gives us that for free; with SQL you'd need a cron job to clean up expired-window rows. - **Why not SQL:** we don't need joins, multi-row transactions, or durability guarantees for a value that's meaningless a minute from now. Paying for ACID/disk persistence here only slows down the one thing that must be fast.

7. Data model (detailed)

```
Redis keys (per algorithm — pick one per deployment):

Fixed window counter:
  key:  ratelimit:{clientId}:{endpoint}:{windowStartEpochMin}
  val:  INTEGER (count so far)
  TTL:  window length (e.g. 60s) — key self-deletes after window ends

Token bucket (see #9 for detail):
  key:  bucket:{clientId}:{endpoint}
  val:  HASH { tokens: FLOAT, last_refill_ts: TIMESTAMP }
  TTL:  refreshed on each access; expires if client goes idle

Sliding window counter (hybrid, cheap approximation):
  key:  ratelimit:{clientId}:{endpoint}:{currWindow}
  key:  ratelimit:{clientId}:{endpoint}:{prevWindow}
  -> weighted sum of the two gives a smoothed "sliding" estimate

Config (rules), stored separately, low write volume — can be a small
SQL table or config service, cached locally on every gateway node:
  rules: (endpoint, client_tier) -> (limit, window_secs, burst_capacity)
```

Why no long-lived "request log" table: storing every request would be the sliding-window **log** algorithm (see #9) — correct but O(requests) in memory. At our scale that's too much data for too little benefit; we approximate instead.

8. Load balancer — L4 vs L7 vs API Gateway

```
  Client
    |
 [ L7 LB / API Gateway ]  <-- rate limiter usually LIVES HERE,
    |                          as gateway middleware/plugin
    |    (auth, TLS termination, routing... then:)
    |    [ Rate Limiter check ] --- allow? ---> forward to service
    |                             \-- deny? ---> 429 right here,
    |                                            never hits backend
    +--> Service A
    +--> Service B
```

Choice: the rate limiter is a plugin/middleware inside the L7 API Gateway, not a separate tier the request has to detour through. This directly reuses the foundations from problem

1: L7 is where per-request decisions happen (path routing, auth, and now rate limiting)

because only L7 has parsed enough of the request (client identity, endpoint, headers) to make the call. **Why not L4:** L4 only sees IPs/ports — no notion of `clientId` or API key, so it can't apply per-user or per-endpoint rules. Rejecting at the gateway, before the request ever reaches a backend service, also protects the backends from load — that's the whole point.

9. Caching + data flow (DB ↔ cache)

This is the crux of the design: **where does the counter actually live, and how "remote" is each check?**

```
Option A — fully remote (every check hits Redis):
  gateway node -> Redis Lua script -> allow/deny
  + always globally accurate
  - every request pays a network round trip (~0.5–1ms) to Redis

Option B — local token bucket per node, no shared state:
  each gateway node keeps its own in-memory bucket for a client
  + zero extra latency (pure in-process check)
  - N nodes each think the client gets the FULL limit -> effective
    limit becomes limit * N (wrong if client's traffic is spread
    across nodes, which it usually is behind a LB)

Option C — hybrid (what's used in practice):
  each node keeps a LOCAL cache of "tokens remaining" for hot
  clients, refilled from Redis periodically (e.g. every 100ms) or
  lazily, and reconciles with Redis on the actual check via the
  atomic Lua script. Local cache absorbs repeated reads; Redis
  remains the single source of truth for the increment itself.
```

Recommendation: default to **Option A (centralized Redis check via Lua script)** for correctness — the 0.5-1ms is usually acceptable since Redis is co-located in the same AZ/rack. Move to **Option C** only when profiling shows the Redis round trip is the actual bottleneck (e.g., ultra-high-QPS internal services) — and explicitly call out the tradeoff: local caching means a node can allow slightly *more* than the global limit for a brief window (eventual consistency of the limit itself) in exchange for lower latency.

10. Concurrency — the centerpiece

The race: two gateway nodes both read "current count = 99, limit = 100" at the same instant, both conclude "one more is fine," both increment, and both allow a request — now 102 requests got through, or worse, both allowed the *same* 101st request. A naive `GET counter; if counter < limit: SET counter+1` is **not atomic** — the check and the increment are two separate round trips with a race window between them.

Fix: make check-and-increment one atomic operation using a Redis Lua script.

```
EVAL "
  local count = redis.call('GET', KEYS[1])
  if count == false then
    redis.call('SET', KEYS[1], 1, 'EX', ARGV[2])
    return 1
  end
  count = tonumber(count)
  if count < tonumber(ARGV[1]) then
    redis.call('INCR', KEYS[1])
    return 1   -- allowed
  else
    return 0   -- denied
  end
" 1 ratelimit:client123 100 60
```

Redis executes the **entire script single-threadedly** — no other client's command can interleave in the middle of it. That's what makes "read, compare, increment" effectively one indivisible step across every node in the fleet, closing the race. (`INCR` + `EXPIRE` alone is atomic *per command* but the check against the limit still needs the script to avoid the read-then-write gap.) For token bucket specifically, the script does "compute elapsed time since last refill, add tokens, cap at bucket size, subtract one if available" — all inside one `EVAL`.

```
Token bucket (visual):

  capacity: 5 tokens, refill rate: 1 token / 12s
  [#####]  bucket full -> request arrives -> take 1 token
  [####.]  4 left
  ... time passes, refill adds tokens back (up to capacity) ...
  [#....]  only 1 left -> next request denied until refill
```

Why token bucket / sliding window counter over the alternatives: | Algorithm | Memory | Accuracy | Burst handling | Verdict | |---|---|---|---|---| | Fixed window | O(1)/key | Poor (2x burst at window edge) | None | simplest, edge-case prone | | Sliding window log | O(requests) | Exact | Exact | too much memory at scale | | Sliding window counter | O(1)/key | Good approximation | Good | **recommended default** | | Token bucket | O(1)/key | Good | **Naturally models bursts** | **recommended when bursts matter** |

Recommendation: token bucket when short bursts should be tolerated (typical API traffic); sliding window counter when you want smooth, memory-cheap approximation without burst allowance. Avoid fixed window (the classic "double burst at the minute boundary" bug) and sliding window log (memory blows up) except for small, high-value cases.

11. Replication — needed?

Yes — but replication interacts badly with atomicity if you're not careful. - Run **Redis in a replicated/clustered mode** (primary + replicas, or Redis Cluster with sharding by `clientId` hash) for availability and to spread read load for local-cache refreshes. - **Writes (the actual increments) must go to the primary** — reading a stale replica for the check-and-increment would reintroduce the exact race from #10 (replica lag means two regions could both think there's quota left). - **On primary failover**, an in-flight counter can be lost or reset if it hadn't replicated yet — for a few seconds, a client might get an unintended "fresh" quota. This is judged acceptable: a rate limiter's job is to prevent *sustained* abuse,

not to be a perfectly precise ledger. Losing a few seconds of count on a rare failover is a reasonable trade for not making the whole system depend on synchronous cross-node writes.

12. Multi-geo

```
Region: US                    Region: EU
[Gateway nodes] -> [Redis-US]  [Gateway nodes] -> [Redis-EU]
       (per-region counters, independent limits)

vs. a single GLOBAL counter (cross-region Redis or a global DB):
  every check pays cross-region latency (50-150ms) -> unacceptable
  for a per-request hot path.
```

Choice: per-region counters, not one global counter. A client's "100 req/min" becomes, in effect, "100 req/min *per region* it happens to hit" — a client hitting both US and EU could get ~2x the intended limit. This is called out explicitly as a **deliberate tradeoff**: perfect global accuracy would require a synchronous cross-region round trip on every request, which defeats the entire purpose of a rate limiter (protecting latency/capacity). Mitigations if global accuracy matters more: (a) route each client consistently to one home region (sticky by clientId), or (b) asynchronously aggregate per-region counts into a global view for *monitoring/alerting*, while still enforcing the (looser) per-region limit in real time.

13. Failure handling

The single most important interview question in this problem: what happens when Redis is down?

```
Redis unreachable/timeout during a check:
  Option 1: FAIL OPEN  -> allow the request.
    + availability: a Redis outage doesn't take down the whole API
    - defenseless against abuse/DDoS exactly when it's most likely
      (an outage is also when systems are most fragile)
  Option 2: FAIL CLOSED -> deny the request.
    + safety: never lets unlimited traffic through
    - a Redis blip now takes down 100% of legitimate traffic too;
      the rate limiter becomes a single point of failure for the
      whole API
```

Recommendation (state it explicitly, don't dodge it): fail open, with a short local fallback limiter — while Redis is unreachable, gateway nodes fall back to a conservative **local, per-node token bucket** (Option B from #9) using the last known limit, rather than either extreme. This bounds the damage (each node still enforces *something*) without letting one dependency's outage cascade into a total API outage. Pure fail-closed is only justified when the protected resource is fragile enough that *any* unlimited traffic is worse than denying everyone (e.g., protecting a payment provider from overage charges).

```
Other failures:
Redis primary dies      -> promote a replica (see #11); brief
                           counter reset is accepted risk.
One gateway node dies   -> LB stops routing to it; no rate-limit
                           state was node-specific to lose (Redis
                           is the source of truth).
Lua script bug/timeout  -> treat as "Redis unreachable" -> fail-open
                           fallback path, not a silent hang.
```

14. Bottlenecks & scaling

- **Redis fan-in at high QPS** → shard by `clientId` hash across a Redis Cluster; each shard independently handles a slice of clients, so throughput scales horizontally.
- **Hot clients** (one very chatty API key) → that client's key lives on one shard; if it dominates, either give it its own dedicated shard or push it toward the local-cache hybrid (#9) so most of its checks never leave the gateway node.
- **Config lookups** (which rule applies to this endpoint/tier) → cache the rule table locally on every gateway node and refresh on a slow interval (seconds), never look it up synchronously per request.
- **Cross-region growth** → add a Redis cluster per region (#12) rather than scaling one global cluster — keeps the hot-path round trip local.

15. Tradeoffs & what to say

> "The core challenge is that many gateway nodes share one logical counter, so a naive read-then-write races. I make check-and-increment atomic with a Redis Lua script — Redis is the right store because it's in-memory, has native TTLs, and gives me that atomicity for free. I'd default to token bucket for its burst tolerance, backed centrally in Redis, with an optional local-cache hybrid if the extra network hop becomes the bottleneck. Replication gives availability but I accept a few seconds of counter loss on failover as a reasonable tradeoff. The hardest question is what to do when Redis itself is unreachable — I'd fail open with a conservative local fallback bucket rather than either taking down the whole API (fail closed) or going fully unprotected (naive fail open)."

Follow-ups an interviewer may probe - How would you rate-limit by IP *and* by API key simultaneously? (Two independent checks, both must pass — most restrictive wins.) - How do you avoid the "thundering herd" when a popular client's bucket refills exactly on a boundary? (Token bucket's continuous refill avoids the fixed-window edge-burst problem.) - How would you let a client see their remaining quota without consuming it? (A read-only `GET` / `peek` path, separate from the atomic decrement script.) - How do you rate-limit at the CDN/edge before it even reaches your gateway? (Push coarse limits to edge nodes with their own local counters; treat as best-effort, refine at the gateway.)

---

4. Distributed Key-Value Store

The problem in one line. Store opaque `key -> value` pairs across many machines so that `get` / `put` / `delete` stay fast and available even when nodes fail or the network partitions (think Dynamo / Cassandra / Riak).

---

1. Requirements

Functional - `put(key, value)` — write/overwrite a value for a key. - `get(key)` — return the value(s) for a key. - `delete(key)` — remove a key (usually a tombstone, not an instant erase). - No queries, no joins, no schemas — the value is opaque bytes to the store.

Non-functional - **Horizontally scalable** — add nodes to hold more data / serve more QPS, without downtime. - **Highly available** — a write should succeed even if some replicas are unreachable. - **Low, predictable latency** for single-key ops (single-digit ms). - **Tunable consistency** — caller can trade latency for correctness per request. - Tolerates node failure, disk failure, and network partitions gracefully (no data loss).

2. Estimation

```
Assume 1B keys, avg value 1 KB, replication factor N=3.
  raw data      = 1B * 1KB           = 1 TB
  replicated    = 1TB * 3            = 3 TB total across the cluster
QPS: 50k reads/sec, 10k writes/sec (read-heavy is typical, but not assumed).
Per-node capacity (SSD, RAM cache): ~500 GB useful, ~5k ops/sec/node.
  nodes for storage  = 3TB / 500GB  ~= 6 nodes (round up, leave headroom -> ~12-20)
  nodes for QPS      = 60k / 5k     ~= 12 nodes
-> a modest cluster of ~20-30 nodes handles this; each key's 3 replicas
   spread the load, and we can add nodes linearly as data/QPS grow.
```

3. API style — REST vs gRPC vs GraphQL

Choice: gRPC (or a lean custom binary protocol) between clients/coordinators and nodes.

```
rpc Get(GetRequest{key})               -> GetResponse{values[], version}
rpc Put(PutRequest{key, value, ctx})   -> PutResponse{version}
rpc Delete(DeleteRequest{key})         -> DeleteResponse{}
```

Why gRPC: this is an *internal*, extremely latency-sensitive, high-QPS protocol between a client library (or coordinator) and storage nodes — binary framing, HTTP/2 multiplexing, and typed contracts beat JSON/REST's parsing overhead at this volume. **Why not REST:** the per-call overhead (headers, JSON) matters when you're doing tens of thousands of ops/sec per node. **Why not GraphQL:** there's nothing to selectively fetch — a KV op is already minimal; GraphQL solves an over-fetching problem this API doesn't have. (A REST/JSON facade can sit in front for external consumers who don't need the last drop of latency.)

4. Idempotency — needed?

Yes, and it shapes the design. - `get` / `delete` are naturally idempotent (delete just leaves a tombstone; repeating it is a no-op). - `put` retries are the tricky one: a client that times out and retries a `put` must not create **two conflicting versions** that look like a genuine concurrent write. Attach a **client request ID** or reuse the same **version/vector-clock context** the client read before writing, so a retried `put` is recognized as "the same write" rather than a new conflicting one. **Say this tradeoff aloud** — it's exactly the kind of subtlety interviewers probe.

5. Network protocol

TCP with a binary RPC framing (gRPC/HTTP2, or a custom protocol like Cassandra's). No need for WebSockets — this is pure request/response, not server push. Persistent connections between clients/coordinators and nodes (connection pooling) avoid TCP/TLS handshake cost on every op. Internally, nodes also gossip over UDP/TCP for cluster membership (see §13).

6. Database — SQL vs NoSQL

Choice: NoSQL — this problem *is* the database. We're building the storage engine itself, so "SQL vs NoSQL" really means: what does each **node** use locally to persist data?

Why NoSQL semantics: the access pattern is pure single-key `get` / `put` / `delete` — no joins, no cross-key transactions, no secondary indexes required. That is precisely the case NoSQL / KV engines optimize for: partition by key, O(1) lookup, horizontally scalable. **Why not SQL:** a relational engine's strengths (joins, multi-row ACID transactions, secondary indexes) are unused here and its coordination overhead would hurt exactly the write availability we need. Each node typically runs an **LSM-tree** engine locally (like LevelDB/RocksDB) — fast sequential writes, background compaction — the same idea used inside Cassandra/Riak/Dynamo.

7. Data model (detailed)

```
Conceptually: key (opaque bytes) -> value (opaque bytes) + metadata

Per local node storage (LSM-tree, e.g. RocksDB):
-------------------------------------------------------------
 key           (BYTES)     what the app looks up by
 value         (BYTES)     opaque blob, store doesn't parse it
 version       (VECTOR CLOCK or TIMESTAMP)   for conflict detection
 tombstone     (BOOL)      marks a delete, kept for an anti-entropy window
-------------------------------------------------------------

No secondary indexes, no schema. If the app needs to query by value or
attribute, that's a different problem (build a separate index, or use a
different store) — this store's one job is fast, available single-key access.
```

8. Load balancer — client-side routing vs coordinator node

There's no traditional L4/L7 load balancer in the hot path — routing is **ring-aware** (depends on `hash(key)`, see §9), not round-robin. Two ways to get a request to the right node:

```
OPTION A: client-side routing (client knows the ring)
  Client library caches the ring layout (nodes + their hash ranges) and
  computes hash(key) locally, then talks DIRECTLY to the owning replica.
    + no extra hop, lowest latency.
    - client library must track ring changes (via gossip feed or refresh).

OPTION B: any node acts as coordinator
  Client sends get/put to ANY node (simple round-robin LB); that node
  computes hash(key), forwards to the true replicas, collects W/R acks,
  and replies to the client.
    + simple, dumb client — no ring logic needed.
    - one extra network hop (client -> coordinator -> replica).
```

Choice: client-side routing for latency-sensitive internal callers (this is what real Dynamo-style stores do — a "smart client"), **falling back to any-node coordination** for external/lightweight clients that don't want to embed ring logic.

9. Partitioning — Consistent hashing (THE CENTERPIECE)

The core problem: with N nodes, naive `hash(key) % N` reshuffles almost **every** key when N changes (a node joins or dies) — a massive, unnecessary data migration. **Consistent hashing** fixes this: only `~1/N` of keys move when the cluster changes size.

The hash ring: nodes and keys are hashed onto the same circular space `[0, 2^32)`. A key belongs to the **first node clockwise** from its hash position.

```
                length ~ 2^32, wraps around to 0
                      ___________
                A45           B  110
               /                    \
         key77 *                     * key130
             /                        \
          key20                       C 200
             \                        /
              \_____________________/
                        key250 *
               (key250 wraps past C back to node A)

key77  -> next node clockwise -> B(110)
key130 -> next node clockwise -> C(200)
key250 -> wraps around        -> A(45)
```

Adding a node D at position 160 — only the slice between B and the old C moves:

```
Before: ... B(110) ------------------[C(200)] ...
After:  ... B(110) ------[D(160)]---[C(200)] ...

Only keys that hashed between 110 and 160 move — from C to D.
Every other key (owned by A, B, or the rest of C's old range) is untouched.
-> exactly the "~1/N keys move" property.
```

Virtual nodes (vnodes): with only one point per physical node, load is lumpy (some nodes get a much bigger arc than others, and adding one node only unloads its immediate neighbor). Fix: each physical node claims **many points** on the ring (e.g. 100-256 vnodes).

```
Physical node A owns vnodes: A1, A2, A3, ... A100  (scattered around the ring)
Physical node B owns vnodes: B1, B2, B3, ... B100
...
-> load spreads evenly across nodes.
-> a NEW physical node claims ~100 small vnode ranges taken from MANY
   existing nodes (not just one neighbor) -> rebalancing work spreads
   across the whole cluster instead of overloading one node.
```

Why this matters for the interview: consistent hashing + vnodes is *the* mechanism that makes this store horizontally scalable with minimal, evenly-spread data movement on membership changes — it's the one diagram to draw clearly and narrate.

10. Concurrency — conflict resolution

Concurrent writes to the same key are the central concurrency problem here (not row locks — there are no cross-key transactions). Two clients `put` the same key at nearly the same time, or during a partition, and different replicas may end up disagreeing:

```
A) Last-Write-Wins (LWW): attach a timestamp, newest timestamp wins.
     + simple, no extra bookkeeping.
     - can SILENTLY drop a legitimate concurrent write (clock skew risk).

B) Vector clocks: each write carries a per-node counter, e.g. [A:2, B:1].
     - if one version's counters are all >= the other's -> it's a
       strict descendant, safe to keep and discard the older one.
     - if neither dominates (e.g. [A:2,B:1] vs [A:1,B:2]) -> genuine
       concurrent write -> return BOTH ("siblings") to the application
       and let it merge them (e.g. shopping-cart union), like Dynamo does.
```

Choice for this design: vector clocks (or a simpler client-supplied version token) when correctness of concurrent edits matters; LWW when simplicity is fine and losing a rare concurrent write is acceptable (e.g. caching, session data). Writes to *different* keys never contend — they route to independent (possibly overlapping) replica sets and proceed in parallel.

11. Replication, quorum, and the CAP tradeoff

Once `hash(key)` locates the key on the ring (§9), **replicate to the next N-1 nodes clockwise** (N = replication factor, commonly 3):

```
N=3: key hashes between B and C -> owned by C (the "coordinator" for this key).
     Replicas also written to the next two nodes clockwise: D, A.

     ... B --- [key] --- C(replica 1) -- D(replica 2) -- A(replica 3) ...
```

Quorum — W, R, N: - **N** = number of replicas for a key. - **W** = number of replicas that must ACK a write before it's considered successful. - **R** = number of replicas that must respond to a read before it's returned. - **Rule of thumb: W + R > N** guarantees every read overlaps with the most recent write on at least one replica → "read-your-writes"-ish strong-ish consistency.

```
Example: N=3, W=2, R=2   (W+R=4 > N=3  -> overlap guaranteed)
  put(): write sent to all 3 replicas, return success once 2 ACK.
  get(): read sent to (at least) 2 replicas, compare versions, return newest.
  -> tolerates 1 slow/dead replica on both paths, still consistent-ish.

Tunable per request:
  W=1, R=1  -> fastest, most available, weakest consistency ("eventual").
  W=N, R=1  -> reads fast & cheap, writes must reach everyone (less available).
  W=1, R=N  -> writes fast, reads check everyone (catches stale replicas).
```

CAP: this is an AP system. Under a network partition, we choose **Availability** over strict **Consistency** — a `put` still succeeds on whichever replicas are reachable rather than blocking, and we accept brief **eventual consistency** (§10 handles the resulting conflicts) instead of refusing writes like a CP system would.

```
CAP triangle: can only fully have 2 of 3 during a partition.
  CP systems (e.g. a consensus config store) refuse writes during a
    partition to stay consistent -> lower availability.
  AP systems (this store) keep accepting reads/writes on both sides of a
    partition -> stay available, resolve divergence later.
```

This tunability — pick your point on the latency/consistency spectrum **per call** — is the signature feature of a Dynamo-style store.

12. Multi-geo

```
Multi-datacenter replication: N replicas are spread across DCs, e.g.
N=3 as "2 in DC1, 1 in DC2" (a rack/DC-aware replication strategy),
so losing an entire datacenter still leaves a majority available.

LOCAL_QUORUM: quorum (W/R) counted only within the LOCAL datacenter,
not across the WAN.
  -> writes/reads don't wait on slow cross-DC round trips for normal
     ops; the write still replicates to other DCs asynchronously.
  -> tradeoff: a DC-local failure can be masked, but a full-DC outage
     briefly reduces the effective replica count elsewhere until that
     DC's copies resync.
```

This is exactly how Cassandra offers `LOCAL_QUORUM` vs `EACH_QUORUM` vs `QUORUM` — pick the consistency scope that matches whether you care more about cross-DC correctness or latency.

13. Failure handling

```
Replica temporarily down at write time (but W still met by others):
  -> HINTED HANDOFF: another node stores the write with a "hint" for the
     down replica, and forwards it once that replica comes back online.

Replicas silently drift out of sync (missed writes, no crash detected):
  -> READ REPAIR: on a get(), if replicas disagree, return the newest
     version to the client AND push it to the stale replica(s) right then.
  -> ANTI-ENTROPY (Merkle trees): each node periodically builds a Merkle
     tree (hash tree) of its key ranges and compares tree hashes with its
     replica peers. Mismatched subtrees -> only that slice of keys needs
     syncing, not a full-dataset diff. Catches drift read-repair never
     touches (cold keys nobody reads).

Node membership changes (join/leave/crash):
  -> GOSSIP PROTOCOL: nodes periodically exchange "who's alive and where"
     state with a few random peers; membership info spreads exponentially
     fast without a central coordinator, and every node eventually learns
     the current ring layout.
```

14. Bottlenecks & scaling

- **Hot key** (one key gets disproportionate traffic) → vnodes only spread *data*, not a single hot key's *load*; mitigate with client-side caching or splitting the hot key's value further upstream.
- **Rebalancing cost** on node join/leave → bounded by consistent hashing + vnodes (~1/N of data, spread across many peers) rather than a full reshuffle.
- **Compaction pressure** on each node's LSM-tree under heavy writes → tune compaction strategy, add nodes to spread write volume thinner per node.
- **Anti-entropy overhead** (Merkle tree comparisons) → run on a schedule / low-priority bandwidth so it doesn't compete with live traffic.
- **Coordinator fan-out** → parallel requests to N replicas, short per-replica timeouts, so one slow replica doesn't stall the whole op past the W/R threshold.

15. Tradeoffs & what to say

> "This is an AP system: I use consistent hashing with virtual nodes to partition keys so that adding or removing a node only moves ~1/N of the data, spread evenly across the cluster. Each key replicates to N nodes; W+R>N gives tunable consistency per call, trading latency for correctness. Under a partition I favor availability — writes still succeed — and resolve the resulting conflicts with vector clocks (or LWW if simplicity matters more than never dropping a concurrent write). Hinted handoff, read repair, and Merkle-tree anti-entropy keep replicas converging without a central coordinator; gossip spreads membership changes. Multi-DC replication with LOCAL_QUORUM keeps normal ops fast while still replicating globally."

Follow-ups an interviewer may probe - Why not just use `hash(key) % N`? (Reshuffles almost every key on any membership change — the whole point of consistent hashing is avoiding that.) - How many vnodes per node, and why not one? (More vnodes = smoother load distribution and rebalancing spread across many peers, at the cost of more bookkeeping; one vnode per node concentrates the join/leave

impact on a single neighbor.) - What if W is never met (too many replicas down)? (Write fails / times out — client can retry or accept a lower W; this is the availability/consistency knob in action.) - LWW vs vector clocks in practice? (LWW is simpler and fine for cache-like data; vector clocks avoid silently losing genuinely concurrent, independently valid writes.)

---

5. Distributed Unique ID Generator

The problem in one line. Hand out globally unique, roughly time-ordered 64-bit IDs to many nodes at high throughput, without a central coordinator on the hot path.

---

1. Requirements

Functional - `nextId()` → returns a 64-bit integer ID, unique across the entire fleet. - IDs should be **roughly sortable by creation time** (newer IDs are numerically larger) — useful for indexing, pagination, and debugging. - Many independent nodes (app servers, DB shards) mint IDs concurrently.

Non-functional - **No single point of contention** — a central "next ID" service that every request calls would become the bottleneck and the outage risk for the whole system. - **Very high throughput per node** (tens of thousands of IDs/sec/node) with **low latency** (a local function call, not a network round trip). - **Compact** (64 bits, fits a DB bigint/index) — unlike a 128-bit UUID. - Tolerate node/network failures without generating duplicates.

2. Estimation

```
Assume 1,000 app nodes, each minting IDs for writes.
  Peak: 50,000 IDs/sec/node during traffic spikes (generous).
  Snowflake reserves 12 bits of sequence per millisecond per node
    -> capacity = 2^12 = 4,096 IDs/ms/node = 4,096,000 IDs/sec/node.
  50,000 << 4,096,000 -> huge headroom per node, no contention.
Total fleet capacity: 1,000 nodes * 4,096,000/sec = ~4B IDs/sec theoretical max.
Timestamp field: 41 bits of milliseconds since a custom epoch
  -> 2^41 ms ~= 69.7 years before it wraps -> plenty for a system's lifetime
     (pick an epoch at launch, not 1970, to maximize headroom).
Worker-id field: 10 bits -> 1,024 unique node identities.
```

3. API style — REST vs gRPC vs GraphQL

Choice: neither, primarily — an in-process library call. The whole point of Snowflake is that a node computes its own ID locally with no network hop. Where a shared network-facing service is needed (e.g., non-JVM clients, or centralizing worker-id logic), expose a small **gRPC** service.

```
In-process:  id = IdGenerator.nextId()          <- no network, sub-microsecond
As-a-service: rpc GenerateId(Empty) -> IdResponse { id: uint64 }   <- gRPC, if needed
```

Why in-process by default: any network call per ID defeats the purpose — it reintroduces a round trip and a shared bottleneck for something that should be a local, lock-free computation. **Why gRPC, not REST, if you must externalize it:** this is an internal, extremely latency-sensitive, high-QPS call between services — gRPC's binary framing and persistent HTTP/2 connections beat REST's per-call JSON overhead. **Why not GraphQL:** there is no flexible query shape here, just "give me one number."

4. Idempotency — needed?

No — and that's expected, not a flaw. `nextId()` is a generator, not a resource write; by definition every call should return a **different** value. There is no "retry the same request and get the same result" contract to honor here (contrast with problem #1's `shorten()`, where determinism was a design choice). If a caller retries after a timeout believing the call failed, it simply gets — and should expect — a brand-new, still-valid ID; the old one, if it was actually generated, is just unused. **Say this explicitly** in an interview: idempotency doesn't apply to pure ID minting, and forcing it (e.g., caching "the last ID for this request") would add complexity for no benefit.

5. Network protocol

None, in the common case — it's a local function call. If exposed as a service (for polyglot clients or to centralize worker-id bookkeeping), use plain **gRPC over HTTP/2 with TLS** for internal traffic: simple request/response, no need for streaming or WebSocket push.

6. Database — SQL vs NoSQL

Choice: essentially none on the hot path — that is the entire point of Snowflake. Unlike a DB auto-increment column or a ticket-server table, generating an ID touches **no database at all**; it's arithmetic on the local clock plus an in-memory counter.

The only persistent coordination need is **worker-id assignment**, and that's not a general-purpose database problem — it's a job for a coordination service:

```
ZooKeeper (or etcd/Consul) — used ONLY at node startup, not per-ID:
  node boots -> registers an ephemeral znode under /workers/
            -> claims the lowest free integer in [0, 1023] as its worker id
            -> caches that worker id in memory for the process lifetime
  (contrast: DB auto-increment or a "ticket server" table WOULD be hit
   on every single ID request -> exactly the bottleneck we're avoiding)
```

7. Data model — the Snowflake bit layout (the centerpiece)

A Snowflake ID is a single 64-bit integer, laid out as four fields:

```
 64 bits total, most-significant bit first:

  1 bit        41 bits                  10 bits        12 bits
+---+-----------------------------+----------------+--------------+
| 0 |   timestamp (ms since epoch)|   worker id    |   sequence   |
+---+-----------------------------+----------------+--------------+
 sign         bits 63-23              bits 22-13      bits 12-0
 (always 0,   milliseconds since      which node      counter,
  keeps ID    a custom epoch          generated       reset to 0
  positive)   e.g. 2024-01-01         this ID         every ms,
              -> ~69.7 years          (0-1023)        incremented
              before wraparound                       per ID minted
                                                       in that ms
                                                       (0-4095)
```

Field-by-field: - **Sign bit (1 bit):** always `0`. Keeps the ID a positive signed 64-bit integer — avoids surprises in languages/DBs where signed types are the default (Java `long`, SQL `bigint`). - **Timestamp (41 bits):** milliseconds since a **custom epoch** (e.g., "2024-01-01") rather than Unix epoch — maximizes the usable range. 2^41 ms ≈ 69.7 years. This field is *why* IDs are roughly time-ordered: as time moves forward, this field only grows, so later IDs are numerically larger (within clock-skew caveats — see §13). - **Worker/machine ID (10 bits):** identifies which node minted the ID. 2^10 = 1,024 possible workers. Assigned once at startup (see §7 data model / §13 failure) — this is what lets 1,024 independent nodes generate IDs with zero coordination between them at request time. - **Sequence (12 bits):** a per-millisecond, per-node counter. Resets to 0 at the start of each new millisecond, increments for every ID minted within that same millisecond. 2^12 = 4,096 — so **one node can mint 4,096 unique IDs within a single millisecond** (= 4,096,000/sec) before it must wait for the next millisecond tick.

Resulting capacity: `4,096 IDs/ms/node × 1,024 nodes = ~4.19M IDs/ms fleet-wide`, i.e. billions per second across the fleet, entirely without coordination, for ~69.7 years.

```
Generation algorithm (per node, conceptually):
  on nextId():
    now = current_time_ms()
    if now < last_ms:                 # clock moved backwards! see §13
      handle_clock_skew()
    if now == last_ms:
      seq = (seq + 1) mod 4096
      if seq == 0:                    # exhausted this ms's budget
        wait_until_next_ms()
        now = current_time_ms()
    else:
      seq = 0
    last_ms = now
    return (now << 22) | (worker_id << 12) | seq
```

8. Load balancer — L4 vs L7 vs API Gateway

Mostly not applicable — there's no "ID generator server" to load-balance to; each app node generates its own IDs in-process. If a **standalone gRPC ID service** is used instead (e.g., for non-JVM/legacy clients that can't embed the library), then a standard **L7/gRPC- aware load balancer** distributes calls across a pool of ID-service replicas — but note this reintroduces a network hop and a shared dependency, which is exactly what embedding the generator in each app node avoids. Prefer the library approach; fall back to a service only when you must.

9. Comparing approaches (and why Snowflake wins)

```
A) UUID (v4, random)
     + zero coordination, works offline, no clock dependency
     - 128 bits (2x the storage/index cost of a bigint)
     - NOT time-ordered -> random insert order kills B-tree index
       locality (bad for DB primary keys at scale)

B) DB auto-increment (single sequence in one DB)
     + trivially unique, trivially ordered
     - single point of contention: every insert serializes through
       one DB/sequence -> caps write throughput, single point of failure
     - doesn't scale horizontally at all

C) Ticket server / range allocation
     + removes the DB write from the hot path most of the time
     - each node periodically asks a central server for a BLOCK of
       IDs (e.g., "give me [50000-59999]") and hands them out locally
     - central server is still a dependency (though hit rarely, not
       per-ID) and a restart/crash can burn/skip an unused range
     - ordering across nodes is coarse (whichever node got which block)

D) Snowflake (recommended)
     + no network call per ID -> local, lock-free, extremely fast
     + roughly time-ordered (good for DB index locality, sorting)
     + compact 64-bit integer
     - needs a real-ish clock and a worker-id assignment mechanism
     - throughput per node capped at 4,096/ms unless you widen the
       sequence field or wait for the next ms
```

Recommendation: Snowflake. It's the only option that is simultaneously coordination-free on the hot path, compact, and time-ordered — the other three each give up one of those.

10. Concurrency

The only shared mutable state is **on a single node**: the `(last_ms, sequence)` pair used to hand out up to 4,096 IDs within the same millisecond.

```
Multiple threads on ONE node calling nextId() concurrently:
  -> guard (last_ms, sequence) with a lock, or better, do the whole
     read-check-increment as one atomic compare-and-swap loop
  -> this is the ONLY contended resource in the whole design, and
     it's in-memory (no network, no disk) -> contention is nanoseconds,
     not milliseconds
Across nodes: NO shared state at all -> no cross-node concurrency
  concerns, because each node owns a disjoint worker-id.
```

This is the crux of why Snowflake scales: it moved the *only* point of contention from a network-shared resource (a DB row/sequence) to an in-memory, per-process one.

11. Replication — needed?

Not in the traditional sense — there's no data store to replicate; the "state" is ephemeral (current time + a counter) and lives in each node's memory. What *does* need resilience: - **Worker-id assignment**

metadata in ZooKeeper/etcd should itself be replicated (these systems use Raft/ZAB internally) so a coordinator crash doesn't strand new nodes at startup. - If nodes crash and restart, they simply request a (possibly new) worker id — no ID history needs to be recovered, because IDs already handed out don't need to be remembered anywhere.

12. Multi-geo

```
Each region gets a disjoint SLICE of the 10-bit worker-id space, e.g.:
  us-east:  worker ids 0-255
  eu-west:  worker ids 256-511
  ap-south: worker ids 512-767
  (reserve the rest for growth / spare nodes)
-> guarantees global uniqueness across regions with ZERO cross-region
   coordination or network calls between regions.
```

Time ordering across regions is only approximate, not exact: each region's clock is independently synchronized (via NTP), so clock drift between regions (typically single-digit milliseconds) means an ID minted in `eu-west` at time T could be numerically slightly smaller or larger than one minted in `us-east` at a true wall-clock time very close to T. This is usually acceptable — "roughly time-ordered, globally unique" is the promise, not "linearizable across continents." If strict global ordering is required, that's a much harder (and slower) problem — e.g., a logical clock or a single ordering authority — and conflicts with the "no coordination" goal.

13. Failure handling

```
Clock moves BACKWARDS on a node (NTP correction, VM pause/resume):
  -> if now < last_ms: the node MUST NOT mint an ID with a smaller
     timestamp than one it already issued (would break ordering AND
     risk a duplicate if sequence also happens to collide).
  -> options: (a) refuse to generate IDs and raise an alarm until the
     clock catches back up to last_ms (safest, small availability hit);
     (b) briefly SLEEP until the clock catches up (works for small
     skew, e.g., a few ms); avoid silently reusing timestamps.
  -> mitigate at the source: run NTP in slew mode (gradual correction),
     not step mode (instant jump), so backward jumps are rare/tiny.

Worker-id collision (two nodes think they own the same worker id):
  -> catastrophic: both nodes can mint IDENTICAL IDs. Prevent via
     ZooKeeper ephemeral znodes: a node's claim on a worker id is tied
     to its session, and ZK guarantees no two sessions hold the same
     znode. If a node crashes, its ephemeral znode disappears and the
     id becomes claimable again only after the session truly expires
     (no split-brain double-claim).

ZooKeeper/etcd is down:
  -> only affects NEW nodes trying to START UP and claim a worker id;
     already-running nodes keep minting IDs fine (worker id is cached
     in memory, no per-ID dependency on ZK). Design principle: the
     COORDINATION path failing must not affect the HOT (generation)
     path — exactly mirroring problem #1's "reads survive write
     failures" principle.

Sequence overflow within a millisecond (>4,096 requested):
  -> node busy-waits (spins or sleeps) for the next millisecond tick,
     then resets sequence to 0. A brief, sub-millisecond stall, not
     an error.
```

14. Bottlenecks & scaling

- **Per-node throughput ceiling (4,096 IDs/ms)** → if a single node genuinely needs more, either accept a sub-ms stall-and-retry, or widen the sequence field (trade timestamp or worker-id bits for it) — a design-time tradeoff, not a runtime fix.
- **Worker-id space (1,024 slots)** → if the fleet grows past ~1,024 nodes, widen the worker-id field (borrow bits from sequence or timestamp) before launch; retrofitting bit widths later breaks existing ID semantics, so size this generously up front.
- **Coordinator (ZooKeeper) load** → trivial; it's touched only at node startup, not per ID, so it never becomes a throughput bottleneck even at large fleet sizes.
- **Clock accuracy** → the real dependency to watch is NTP health across the fleet, not CPU or network capacity.

15. Tradeoffs & what to say

> "I'd use Snowflake-style IDs: a 64-bit value packing a 41-bit millisecond timestamp, a 10-bit worker id, and a 12-bit per-millisecond sequence — generated as a local, in-process computation with no network call and no database on the hot path. That's the key

> advantage over a DB auto-increment or a ticket server, both of which put a shared, network-visible resource in the critical path. UUIDs avoid coordination too but are 128 bits and not time-ordered, which hurts index locality. The only coordination I need is a one-time worker-id claim via ZooKeeper at node startup, using ephemeral znodes to guarantee no two nodes ever claim the same id. The one real hazard is clock skew — if a node's clock jumps backward, I either stall until it catches up or refuse to generate rather than risk duplicate/out-of-order IDs."

Follow-ups an interviewer may probe - What if you need IDs sorted with byte-order string comparison (e.g., Cassandra time buckets)? (Encode as a fixed-width, zero-padded string, or use a ULID-style format.) - How would you widen capacity without a coordinator? (Shift bit allocation: fewer worker-id bits if the fleet is small, more sequence bits if per-node throughput must be higher.) - Why not just use the database's own auto-increment and shard it (e.g., odd/even ranges per node)? (Works at small scale, but every node must know the full partitioning scheme and re-coordinate on membership change — Snowflake's worker-id space sidesteps this.) - How do you debug "which node/when" from an ID? (Bit-shift and mask it back out — that's a practical benefit of the layout being simple arithmetic, not an opaque hash.)

---

6. Hotel / Flight Booking System

The problem in one line. Let users search available rooms by city/date/guests, then reserve and pay for one — without ever selling the same room twice.

---

1. Requirements

Functional - Search availability: `city + check-in/out dates + guest count` → list of hotels/rooms. - View a hotel/room's details. - Reserve a room for given dates, pay for it, get a confirmation. - Cancel a booking. - Inventory: each hotel has N rooms of a given type available per calendar date.

Non-functional - **Correctness over speed on the write path** — never oversell a room (strict consistency). - **Read-heavy on search** — thousands of browsers per one booker ("funnel" shape). - **Low latency search** (< 300 ms) so results feel interactive. - **Payments must be safe** — no double charge, no charge-without-booking. - Booking write QPS is low compared to search QPS, but **correctness there is non-negotiable.**

2. Estimation

```
Assume 50,000 hotels, avg 100 rooms => 5M room-nights of inventory rows
  per day of the booking window (rolling 365-day window):
  5M rows/day * 365 = ~1.8B inventory rows (fits SQL with partitioning).

Traffic: 10M searches/day, 200k bookings/day (50:1 browse:book ratio).
  search QPS = 10M / 86400 ~= 116/s avg (peak ~5-10x -> ~1,000/s)
  booking QPS = 200k / 86400 ~= 2.3/s avg (peak ~50-100/s)
-> Search is the scale problem; booking is the CORRECTNESS problem.
```

3. API style — REST vs gRPC vs GraphQL

Choice: REST for the client-facing API; gRPC between internal microservices.

```
GET  /api/v1/search?city=SFO&checkin=...&checkout=...&guests=2
GET  /api/v1/hotels/{id}
POST /api/v1/bookings          { roomId, dates, guestInfo }  (Idempotency-Key header)
POST /api/v1/bookings/{id}/pay { paymentMethod }             (Idempotency-Key header)
POST /api/v1/bookings/{id}/cancel

Internally: Search Service --gRPC--> Inventory Service --gRPC--> Booking Service
                                                       --gRPC--> Payment Service
```

Why REST externally: simple resource CRUD, cacheable GETs for search, and every client (web/mobile) speaks HTTP/JSON natively. **Why gRPC internally:** the booking flow fans out across several microservices (inventory check, booking, payment, notification) — low-latency typed contracts matter

more than human-readability once it's service-to-service. **Why not GraphQL:** the search response shape is fairly fixed (a list of hotel cards); no deep client-driven nesting problem to solve.

4. Idempotency — needed?

Yes — this is the centerpiece for this problem, on TWO endpoints. - `POST /bookings` (create/confirm booking) and `POST /bookings/{id}/pay` (charge payment) **must** be idempotent. A client on a flaky network will retry a timed-out "Book Now" or "Pay Now" click. Without protection: retry on booking → **double-booked room**; retry on payment → **double-charged card**. - Client generates an `Idempotency-Key: <uuid>` per user action. Server stores `(idempotency_key -> result)` and, on a duplicate key, **returns the original result** instead of re-executing the side effect.

```
Client click "Pay" -> POST /pay  Idempotency-Key: k1
  request times out (client doesn't know if it succeeded)
  client retries -> POST /pay  Idempotency-Key: k1  (SAME key)
    server: "have I seen k1?" YES -> return stored result, DO NOT charge again
```

- Store the idempotency table in the **same DB/transaction** as the payment/booking write, so "record the key" and "do the side effect" commit or fail together.

5. Network protocol

HTTPS (TLS) everywhere. Search/browse is plain request-response — no WebSocket needed (no server push required for search results). Payment calls go through a PCI-compliant gateway over HTTPS with tokenized card data (we never store raw card numbers). Internal gRPC rides on HTTP/2, which also multiplexes the several internal calls a booking triggers.

6. Database — SQL vs NoSQL

Choice: SQL (Postgres/MySQL) for the transactional core — bookings, inventory, payments. A separate search index (Elasticsearch or a denormalized read store) for browse/search.

Why SQL for the core: the booking flow needs **ACID transactions** across multiple rows (decrement inventory AND insert a booking AND record a payment, atomically), **strong consistency** (you must never let inventory go negative), and **relational structure** (Hotel → Room → Booking → Payment are naturally joined entities). This is exactly the case the SQL/NoSQL framework calls out: relationships + multi-row transactions matter → SQL.

Why not NoSQL for the core: a KV/document store gives you fast single-key reads but no multi-row ACID transaction across "check availability + decrement + create booking" — you'd have to hand-roll locking/compensation, reinventing what a relational DB gives for free.

Why a separate search index: availability search ("hotels in SF, these dates, 2 guests") is read-heavy, filter/sort-heavy, and can tolerate slightly stale data — a perfect fit for Elasticsearch (or a denormalized cache table) fed asynchronously from the SQL source of truth. **Search reads from the index; booking writes go straight to SQL and re-validate there** (see §9 and §11 — this split, and why search staleness is safe, is the key insight).

7. Data model (detailed)

```
Hotel
-------------------------------------------------
 hotel_id (PK), name, city, geo, star_rating, ...

RoomType                                  (a class of room in a hotel)
-------------------------------------------------
 room_type_id (PK), hotel_id (FK), name, capacity, base_price

RoomInventory                              <-- THE CENTRAL TABLE
-------------------------------------------------
 hotel_id, room_type_id, date        COMPOSITE PK
 total_rooms      (INT)              e.g., 20 rooms of this type
 booked_rooms     (INT)              how many are currently held/confirmed
 version          (INT)              optimistic-lock counter
-------------------------------------------------
 available(date) = total_rooms - booked_rooms
 ONE ROW PER (hotel, room_type, date) -> a calendar-sharded inventory.
 Booking a 3-night stay touches 3 rows (one per date), all in one txn.

Booking
-------------------------------------------------
 booking_id (PK), user_id, hotel_id, room_type_id,
 checkin_date, checkout_date,
 status: HELD | CONFIRMED | CANCELLED | EXPIRED
 hold_expires_at  (TIMESTAMP, used while status = HELD)
 idempotency_key  (UNIQUE)
 created_at

Payment
-------------------------------------------------
 payment_id (PK), booking_id (FK), amount, status: PENDING|SUCCEEDED|FAILED,
 idempotency_key (UNIQUE), gateway_ref, created_at
```

Why per-date inventory rows, not just a `rooms_left` counter on the hotel: a stay spans multiple nights and each night can sell out independently (Friday full, Saturday open) — the calendar-sharded design is what makes "is this date range available" a simple range query and what makes locking precise (lock only the affected date rows, not the whole hotel).

8. Load balancer — L4 vs L7 vs API Gateway

```
 Client
   |
[ API Gateway / L7 ]  <- TLS termination, auth, rate limiting,
   |                     path routing to microservices
   +--> Search Service     (read replicas + search index, autoscaled)
   +--> Inventory Service  (talks to primary DB, transactional)
   +--> Booking Service    (talks to primary DB, transactional)
   +--> Payment Service    (talks to external payment gateway)
```

Choice: L7 / API Gateway. We route `GET /search` to a large fleet of read-optimized search instances, and `POST /bookings` / `POST /pay` to a smaller, carefully-guarded transactional fleet — that's path-based routing, an L7 job. The gateway is also where we put **auth** (must be logged in to book) and **rate**

limiting (stop bots from hammering search or hoarding holds). Plain L4 can't make any of these content-aware decisions.

9. Caching + data flow (DB ↔ cache)

Cache what's safe to be stale; never cache the number a booking decision depends on.

```
Hotel details, photos, reviews  -> cache-aside, long TTL (rarely changes).
Search RESULTS (list of hotels + "rooms from $X") -> cache-aside, short TTL,
  OR served from the async-refreshed Elasticsearch index. Approximate is fine
  here — it's just for browsing.

BUT: the actual "is there a room left" decision at booking time
  NEVER reads from cache or the search index. It reads RoomInventory
  from the primary SQL DB, INSIDE the booking transaction. This is the
  one number in the whole system that must always be fresh.
```

Rationale: showing a slightly-stale "3 rooms left" in search results is a minor UX blemish; *deciding* to sell a room based on a stale number is a double-booking. Split "browse" data (cacheable) from "the authoritative write-decision" (never cached) — this is the single most important caching lesson in this problem.

10. Concurrency — preventing double-booking (the centerpiece)

The race: the last room, two users, same moment.

```
User A: read available = 1        User B: read available = 1
User A: available > 0 -> book!    User B: available > 0 -> book!
User A: write booked_rooms += 1   User B: write booked_rooms += 1
-> booked_rooms now exceeds total_rooms. TWO bookings for ONE room.
```

Both read stale data before either writes — a classic **read-then-write race**.

Fix A — pessimistic lock (`SELECT ... FOR UPDATE`):

```
BEGIN;
  SELECT booked_rooms, total_rooms FROM room_inventory
    WHERE hotel_id=? AND room_type_id=? AND date IN (...)
    FOR UPDATE;                          -- locks these rows
  -- check available = total - booked > 0 for EVERY date in the stay
  UPDATE room_inventory SET booked_rooms = booked_rooms + 1 WHERE ...;
  INSERT INTO booking (..., status='CONFIRMED');
COMMIT;                                   -- lock released
-- a concurrent request for the same rows now BLOCKS until this commits,
-- then re-reads the updated (now-zero) availability and correctly fails.
```

Fix B — optimistic lock (version column), no blocking:

```
SELECT booked_rooms, version FROM room_inventory WHERE ...;
-- check available > 0 in application code
UPDATE room_inventory SET booked_rooms = booked_rooms + 1, version = version + 1
  WHERE hotel_id=? AND ... AND version = <version just read>;
-- if 0 rows updated -> someone else won the race -> retry read or fail
```

Recommendation: optimistic locking + a short-lived HOLD. Reserve the room (`status=HELD`, `booked_rooms` incremented, `hold_expires_at = now()+10min`) the moment the user starts checkout, using the version check above so concurrent holds can't both win. This mirrors "seat locking" in ticketing: it gives the user a firm window to enter payment without a pessimistic lock sitting open across a slow human interaction (typing card details), while still guaranteeing only one HOLD can succeed per room. A background job expires unpaid HELD rows back to available. Pessimistic `FOR UPDATE` is fine too and is simpler to reason about, but holding a row lock for the seconds it takes to run the transaction (not minutes for a human to pay) is the key difference — so pessimistic locking covers the "commit the booking" instant, and the *hold* covers the "give the human time to pay" window.

11. Replication — needed?

Yes, read replicas — but with a critical caveat for booking correctness.

```
Primary (writes: booking, payment, inventory decrement)
   |  async replication
   +--> Replica 1  --\
   +--> Replica 2  ---+--> Search Service reads (browse/availability search)
```

- Search/browse traffic hits **read replicas** — high QPS, tolerant of a little staleness.
- Replica lag means **search results can show a room as available when it just sold out on the primary a moment ago.** This is acceptable *only* because we **always re-validate against the primary inside the booking transaction** (§10) before confirming — the replica is a hint for browsing, never the source of truth for the write. If we skipped re-validation, replica lag would directly cause oversells.
- Writes (booking, inventory decrement, payment) always go to the **primary** — no writes on replicas, ever.

12. Multi-geo

```
Each hotel's inventory has a HOME REGION (near where the hotel/chain
operates or where the booking-service shard for that hotel_id lives).
Writes for that hotel's bookings go to its home region's primary DB.

Global search: either
  (a) fan out a search query to every region's index and merge results, or
  (b) maintain one globally-replicated search index (fed async from every
      region's primary) so a user in Tokyo can search European hotels
      from a local, low-latency read-replica of the search index.
Booking a European hotel from Tokyo still writes to Europe's primary
(data locality for the authoritative record) -> slightly higher write
latency for that one request, which is acceptable since booking is rare
compared to search.
```

This mirrors the caching principle at global scale: keep the *cheap, frequent* operation (search) local and fast everywhere; keep the *rare, must-be-correct* operation (booking write) anchored to the data's home region.

13. Failure handling

```
Payment succeeds, booking write fails (crash between the two steps)
  -> use a SAGA / outbox pattern: write booking+payment-intent in ONE
     local transaction first (status=HELD, payment=PENDING), THEN call
     the payment gateway, THEN mark CONFIRMED. If the gateway call
     succeeds but the final "mark CONFIRMED" write fails, a reconciliation
     job compares gateway status vs booking status and fixes it (or
     issues a compensating REFUND if the booking truly can't be honored).
Idempotency key reused (client retry)
  -> return the stored result of the original attempt; never re-charge
     or re-book (see §4).
Hold expires before payment
  -> background sweeper flips HELD -> EXPIRED, decrements booked_rooms,
     frees the room back to inventory automatically.
Search index / Elasticsearch down
  -> fall back to a degraded search (read replicas directly, slower) or
     show "results may be incomplete"; booking path is UNAFFECTED since
     it never depended on the search index.
Payment gateway down
  -> fail fast, keep the HOLD alive (don't lose the user's spot) up to
     its TTL, let the user retry payment; never silently confirm without
     a successful payment.
DB primary down
  -> failover to a promoted replica for writes; brief write unavailability
     is acceptable (bookings pause) but reads (search) continue serving.
```

Design principle: **payment and booking-state changes must be atomic or sagas with compensation** — partial success between "charged" and "booked" is the failure mode that actually loses companies money and trust, so it gets designed for explicitly, not left to hope.

14. Bottlenecks & scaling

- **Search QPS** → scales horizontally: more read replicas, a sharded search index, and aggressive caching of hotel metadata (§9). This is the "many stateless readers" problem.
- **Hot inventory rows** (one very popular hotel, one popular date) → the row-level lock or version-check in §10 becomes a contention point; mitigate by keeping transactions short (lock only the touched date rows, commit fast) and by sharding inventory by `hotel_id` so unrelated hotels never contend with each other.
- **Payment gateway latency** → don't hold the inventory row lock across the external payment call; that's exactly why the HOLD pattern (§10) decouples "reserve" (fast, locked) from "pay" (slow, external, unlocked).
- **Inventory table growth** → rolling 365-day window, partition by date, archive/drop rows for past dates.

15. Tradeoffs & what to say

> "Search is read-heavy and can be approximate, so it's served from replicas and a denormalized search index with caching. Booking is the opposite: low volume but must be exactly correct, so it's a SQL transaction with optimistic locking (a version column) on a per-date RoomInventory row, wrapped in a short-lived HOLD so the user has time to pay without holding a DB lock open. Payment and booking creation are idempotent via a client Idempotency-Key, stored transactionally, so retries never double-charge or double-book. I never let the write-path availability check touch a cache or a stale replica — it always reads the primary inside the transaction, which is what makes replica lag on the search side safe. Multi-geo keeps each hotel's authoritative inventory in its home region while search fans out or reads a globally replicated index."

Follow-ups an interviewer may probe - Why not just use a single `SELECT FOR UPDATE` everywhere and skip the HOLD? (Because it would hold a row lock across a slow human/payment-gateway round trip, killing throughput on popular inventory — separate the fast "reserve" commit from the slow "pay" step.) - How do you pick optimistic vs pessimistic locking? (Optimistic wins under low contention with fast retries; pessimistic is simpler when contention is expected to be high and transactions are short — both are legitimate, name the tradeoff.) - What if the payment gateway is itself not idempotent? (Generate your own idempotency key, pass it to the gateway if it supports one; otherwise track "did I already call this gateway for this booking" locally before calling.) - How do you avoid overselling across a multi-night stay atomically? (Lock/version-check every date row for the stay inside one transaction — all dates succeed or the whole booking fails and rolls back.)

---

7. Airbnb (Listings + Search + Booking)

The problem in one line. Hosts publish listings with photos, amenities, and an availability calendar; guests **search** by location + dates + filters and book — the hard part is fast, relevant geo-search over live availability, not the booking transaction itself (see problem #6 for double-booking depth).

---

1. Requirements

Functional - Host: create/edit a listing (title, photos, amenities, location, price, availability calendar). - Guest: **search** by location (city/map bounds) + check-in/check-out dates + filters (price range, guest count, amenities) → ranked list of available listings. - Guest: view listing detail (photos, host, reviews, availability), then book. - Reviews after checkout (guest ↔ host, brief — not the focus here).

Non-functional - **Search is the hot path**: read-heavy, sub-second latency, must reflect availability reasonably fresh (small staleness tolerable — re-validated at booking time). - **Booking must be strongly consistent** for a given listing+date range (no double-book) — detailed in problem #6; here we just acknowledge the boundary. - High availability for search/browse (revenue depends on it); booking can be slightly less available than search without breaking the product. - Global: listings and guests are spread worldwide → geo-locality matters for latency and data-residency law (GDPR).

2. Estimation

```
Assume 7M active listings, 150M searches/day, 2M bookings/day.
  search QPS = 150M / 86,400 ~= 1,700 QPS avg (peak ~5–8k, very spiky by
               time-zone/season)
  booking QPS = 2M / 86,400  ~= 23 QPS avg (booking is a tiny fraction
               of traffic vs search -> read:write ~ 1000:1)
Storage:
  listing metadata: 7M * ~5 KB (text+attrs) = 35 GB           -> SQL, trivial
  photos: 7M listings * 20 photos * 200 KB   = ~28 TB          -> object storage
  availability calendar: 7M listings * 365 days * ~20B/row    = ~50 GB/yr -> SQL, trivial
  search index (denormalized doc per listing incl. geo)       = few hundred GB, fits
    a sharded Elasticsearch cluster comfortably
```

Takeaway: search traffic dwarfs booking traffic by ~1000:1 — the system is optimized around **read-heavy geo-search**, and booking gets its own narrower, stricter subsystem.

3. API style — REST vs gRPC vs GraphQL

Choice: GraphQL for the client-facing app, REST for simple public endpoints, gRPC internally.

```
GraphQL: query {
  listing(id: "123") {
    title photos(first: 5) { url }
    host { name responseRate }
    reviews(last: 3) { rating text }
    availability(from: "2026-08-01", to: "2026-08-07") { date price available }
  }
}
```

Why GraphQL here (and not in the URL shortener): a listing screen needs listing fields + host profile + a slice of reviews + a slice of availability + photos, all nested and each client (iOS/Android/web) wants a different subset. Without GraphQL you'd either **over-fetch** (one bloated REST response) or make the client fire **4-5 REST calls** (listing, host, reviews, availability, photos) and stitch them — extra round trips hurt on mobile networks. GraphQL lets the client ask for exactly this shape in **one round trip**. This is the classic "many resource shapes, over-fetching pain" case the framework calls out.

Why still REST somewhere: simple, cacheable endpoints (e.g., `GET /listings/{id}/ical`, webhooks) don't need GraphQL's flexibility — plain REST is simpler and more cache-friendly at a CDN. **Why gRPC internally:** the search service, booking service, and pricing service talk to each other at high volume — typed Protobuf + HTTP/2 streaming beats REST/JSON for internal latency.

4. Idempotency — needed?

- **Search is a GET — naturally idempotent**, no key needed.
- **Booking/payment is not** — a retried "confirm booking" request must not create two reservations or charge twice. Client sends an `Idempotency-Key`; server dedupes. This is the same mechanism as problem #6's booking flow — **not re-derived here**, just applied.
- **Listing creation/update**: PUT-style "set listing to this state" is naturally idempotent by resource id; no key needed for edits.

5. Network protocol

HTTPS with HTTP/2 everywhere. HTTP/2 multiplexing matters a lot for the listing-detail GraphQL call and for image-heavy pages (many photo requests over one connection). No WebSocket needed for search/browse (no server push); host-guest messaging (if in scope) would use WebSocket, but that's a separate chat subsystem (see problem #9).

6. Database — SQL vs NoSQL

Choice: SQL (Postgres/MySQL) as the source of truth for listings, availability, and bookings; Elasticsearch as a denormalized, purpose-built search index; object storage for media. Three different stores for three different access patterns — this is the core lesson of this problem.

```
Booking + availability + payments -> need multi-row ACID transactions
  (hold a date range, decrement inventory, charge, commit) -> SQL.
Geo + text + numeric-range search across millions of listings with
  ranking, faceted filters (price/amenities), and "listings visible
  in this map viewport" -> SQL can't do this well (no native geo-
  ranking, filter+sort at this scale gets slow) -> Elasticsearch with
  geo_point / geo_shape fields and a geo-distance / bounding-box query.
Photos, video -> large immutable blobs -> object storage (S3) + CDN,
  never in a row-oriented DB.
```

Why a search index separate from the source-of-truth DB: SQL is optimized for transactional correctness on a *few rows* (one listing's booking). Search needs to scan and rank *millions of rows* by geo-distance + free-text + filters + relevance score in milliseconds — a fundamentally different query shape (OLAP-ish, denormalized, eventually consistent) than OLTP writes. Trying to serve both from one SQL schema means either slow searches or an over-indexed, hard-to-scale-write table. So: **write to SQL (truth), then asynchronously project a denormalized document into Elasticsearch (search).** The tradeoff is staleness — the index can lag the DB by seconds — which is why booking always **re-checks real availability in SQL** at confirmation time (see §13).

7. Data model (detailed)

```
Table: hosts
-------------------------------------------------
 host_id (PK), name, email, response_rate, created_at

Table: listings
-------------------------------------------------
 listing_id (PK), host_id (FK), title, description,
 lat, lng, address, base_price, max_guests,
 amenities (JSON/array), status (active/paused), created_at

Table: availability               <- source of truth for bookability
-------------------------------------------------
 listing_id (FK), date, available (BOOL), price (nullable override)
 PRIMARY KEY (listing_id, date)     one row per listing per night

Table: bookings
-------------------------------------------------
 booking_id (PK), listing_id (FK), guest_id (FK),
 check_in, check_out, status (pending/confirmed/cancelled),
 total_price, idempotency_key, created_at
 -- a booking "claims" a contiguous range of availability rows;
    see problem #6 for the row-locking / transaction details

Table: reviews
-------------------------------------------------
 review_id (PK), booking_id (FK), author_id, rating, text, created_at

Media: photos stored in S3, keyed by listing_id; DB stores only URLs.

Elasticsearch doc (denormalized, rebuilt from the tables above):
{
  listing_id, title, geo_point: {lat, lng}, price, max_guests,
  amenities: [...], avg_rating, photo_url,
  available_dates: [...]   // or a nested availability range check
}
```

`availability` keyed by `(listing_id, date)` is what makes "is this listing free for these dates" a simple range read/write in SQL, and what the search index approximates for fast filtering before the authoritative re-check.

8. Load balancer — L4 vs L7 vs API Gateway

```
  Client (app/web)
       |
  [ GeoDNS ] -> nearest region
       |
  [ L7 / API Gateway ]  <- TLS, auth, rate limit, routing
       |
       +--> Search Service       (GraphQL/REST, read-heavy, autoscaled)
       +--> Listing Service      (CRUD, host-facing)
       +--> Booking Service      (SQL transactions, stricter scaling)
       +--> Media/CDN            (photos)
```

Choice: API Gateway (L7) at the edge. We route by path/operation to independently scaled services — search needs *far* more capacity than booking (§2's 1000:1 ratio), so they must scale separately. The gateway also handles auth (host vs guest permissions), rate limiting (protect search from scraping), and TLS termination. L4 alone can't make path/content-based routing decisions.

9. Caching + data flow (DB ↔ cache)

Search and listing views are extremely read-heavy relative to writes, so cache aggressively at multiple layers.

```
SEARCH read path:
  query (loc+dates+filters) -> Elasticsearch (already a fast, largely
    in-memory-indexed store) -> optionally cache the rendered result
    page in Redis keyed by (geohash-cell, date-range, filter-hash) with
    a short TTL (30-60s) since availability shifts constantly.

LISTING DETAIL read path (cache-aside):
  GET listing/123 -> check Redis -> HIT: return
                                 -> MISS: read SQL (+recent reviews) -> cache -> return
  Photos served from CDN (edge cache), never from the app tier.

WRITE path (host edits listing or availability):
  write SQL -> invalidate Redis key for that listing
            -> emit event -> async job re-indexes the listing doc in
               Elasticsearch (near-real-time, not synchronous)
```

Why not synchronous index update on every write? Search-index writes are more expensive (analysis, geo encoding) and don't need read-your-own-write consistency — a host's price change showing up in search a few seconds late is an acceptable tradeoff for not slowing down every edit or every booking.

10. Concurrency

- **Geo-search itself has no shared-state race** — it's a read query, trivially concurrent.
- **Double-booking** (two guests booking overlapping dates on the same listing) is the classic race here — **fully covered in problem #6** (row-level locks / conditional writes on the `availability` rows inside a transaction). We don't re-derive it; the key point for *this* problem is that the search result showing "available" is only a **hint** — the booking service must re-validate against the authoritative `availability` table before confirming.
- **Calendar updates**: a host blocking dates while a guest is mid-checkout on those same dates → same guard as booking (conditional update / transaction on `availability`).

11. Replication — needed?

Yes, on two independent axes. - **SQL**: leader-follower — writes (bookings, listing edits) go to the primary; read replicas serve listing-detail reads and analytics, tolerating small lag. - **Elasticsearch**: each index shard has replica shards (typically 1-2) for both availability *and* fault tolerance — search reads are load-balanced across replicas. Because search is already "best-effort fresh" (re-validated at booking), replica lag here is a non-issue, unlike a financial ledger.

12. Multi-geo

```
Listings are inherently geo-located -> route search requests to the
region that owns that geography (e.g., search "Paris" hits the EU
region's ES cluster + EU read replica).

[ User ] -> GeoDNS -> nearest region's API Gateway
                        -> Search: query LOCAL Elasticsearch shard set
                           for that region's listings (data locality)
                        -> Booking: writes go to the SQL primary that
                           owns that listing (home region for the host)
```

Benefits: lower search latency (query local index, not cross-ocean), and **GDPR/data residency** — EU host/guest PII and EU listing data can be kept within EU infrastructure by sharding both SQL and the search index by region. Cross-region browsing (a US guest searching Paris) reads from the Paris region's index — a small latency cost, but rare relative to in-region search.

13. Failure handling

```
Search index lags source of truth -> listing looks "available" in
  search but was just booked -> booking service re-checks the SQL
  availability table transactionally before confirming -> if taken,
  return "no longer available," client re-searches. This is the
  central failure mode of a denormalized index and must always be
  assumed, never patched around.
Elasticsearch cluster/shard down -> serve from replica shards; if the
  whole cluster is down, degrade to a simpler SQL-backed "browse by
  city" fallback (worse ranking, but still functional) rather than a
  blank page.
Redis cache down -> fall back to SQL/ES directly (slower, still correct).
SQL primary down -> promote a replica (brief write outage); reads
  (search, browsing) are unaffected since they don't hit the primary.
CDN/object storage blip -> show placeholder images, never fail the
  whole listing page for a missing photo.
```

Design principle: **search is allowed to be approximately right and fast; booking must be exactly right, even if slightly slower** — and the seam between them (the re-validation step) is where that tradeoff is enforced.

14. Bottlenecks & scaling

- **Search QPS** → Elasticsearch shards horizontally by listing (e.g., geo-partitioned shards); add replicas for read throughput; Redis absorbs repeated identical queries.
- **Hot geographies** (popular cities, peak season) → those regions/shards get disproportionate load; shard hot cities further, cache their search results harder.
- **Photo bandwidth** → CDN absorbs almost all of it; origin (S3) rarely touched after first fetch per region.
- **Index rebuild lag under high write volume** (bulk price updates during a sale) → batch and rate-limit the indexing pipeline so it doesn't starve normal reindex traffic.

- **Booking transactions** → much lower volume than search, but see problem #6 for how that narrower system scales its own writes (sharding by listing_id, row-level locking).

15. Tradeoffs & what to say

> "This is fundamentally a **read-heavy geo-search problem** wrapped around a **narrow, strict booking transaction**. I keep SQL as the source of truth for listings, availability, and bookings — because booking needs ACID guarantees — but I denormalize listings into Elasticsearch for geo + text + filter search, because that access pattern is nothing like a transactional lookup. GraphQL fits the client-facing app well since a listing screen needs nested host/photos/reviews/availability in one round trip. The seam between the two stores is the important design decision: search results are a *hint*, and the booking service always re-validates against SQL before confirming, so index staleness never causes a bad booking — it just causes an occasional 'no longer available' retry."

Follow-ups an interviewer may probe - How do you keep Elasticsearch roughly in sync with SQL? (CDC/event stream off the DB → async indexer, not dual writes from the app.) - How do you rank search results? (relevance score blending distance, price, rating, past-booking conversion — a scoring model on top of ES's query.) - What if a guest's dates straddle a host's calendar update mid-search? (Same as double- booking — re-validate at booking time, problem #6.) - How would you support map-viewport search ("search as I drag the map")? (geo_bounding_box query in Elasticsearch, debounced client-side requests.)

---

8. News Feed (Twitter / Instagram)

The problem in one line. Let users post content and follow others, then show each user a fast, roughly-time-ordered feed of everything the people they follow have posted.

---

1. Requirements

Functional - `post(userId, content)` → create a post (text/image/video). - `follow(userId, targetId)` / `unfollow(...)`. - `getFeed(userId, cursor)` → paginated home timeline of followed users' posts, newest first (or ranked — we'll treat reverse-chron as the baseline and mention ranking as an extension).

Non-functional - **Read-heavy**: feed views vastly outnumber posts (~100:1 or more). - **Feed reads must be fast** (< 200 ms) — it's the app's home screen, hit constantly. - **Write availability matters** — a post should never be lost, even if it's delayed. - **Eventual consistency is fine**: a follower seeing a new post 1-2 seconds late is a non-issue. Strong consistency is *not* required. - **Skewed fan-out**: most users have ~100s of followers; a tiny few (celebrities) have millions. This skew is the whole design problem.

2. Estimation

```
Assume 200M daily active users, avg 500 followers/user (celebrities up to 50M).
  posts/day        = 50M            -> ~600 writes/sec avg (peak ~3k)
  feed reads/day    = 200M * 10 opens/day = 2B -> ~23,000 reads/sec (peak ~100k)
  read:write ratio  ~ 40:1  (feed views dominate)

Fan-out cost (push model), if EVERY post fans out to EVERY follower's feed:
  avg case: 600 posts/sec * 500 followers   = 300,000 feed-inserts/sec  (fine)
  celebrity post: 1 post to 50M followers   = 50,000,000 feed-inserts for ONE post
    -> at even 50k inserts/sec of capacity, that's 1,000 seconds (~17 min) just to
       fan out ONE tweet. This is exactly why pure push doesn't work. (Section 3.)

Storage: 50M posts/day * 365 * 5 yrs ~= 90B posts.
  ~1 KB/post metadata (media stored separately) -> ~90 TB metadata over 5 years.
Feed cache: 200M users * 800 cached post-IDs * 8 bytes ~= 1.3 TB in Redis (fits a cluster).
```

3. The centerpiece — fan-out on write vs. fan-out on read

This is the core decision in a feed system: **when do we do the work of assembling a user's feed — at post time, or at read time?**

Fan-out on write (push). When a user posts, immediately push that post into the precomputed feed (a list of post-IDs) of every one of their followers.

```
POST by user A (500 followers)
  A posts
    |
    v
[ Fan-out worker ]  -- for each follower F of A --> prepend postId to
    |                                                Feed(F) in Redis
    v
Feed(follower_1) = [ newPostId, ...older ]
Feed(follower_2) = [ newPostId, ...older ]
...
Feed(follower_500) = [ newPostId, ...older ]

READ: GET Feed(userId) -> Redis list is already built -> O(1) fetch. FAST.
```

- **Pro:** reads are trivial — just read a precomputed list. This is what makes the 40:1 read-heavy workload cheap.
- **Con: write amplification.** One post from a celebrity with 50M followers means 50M writes. Slow, and it can DDoS your own fan-out pipeline (the "celebrity problem" / hot key on write).

Fan-out on read (pull). Store nothing precomputed. When a user opens their feed, look up who they follow, fetch each of those users' recent posts, and merge/sort on the fly.

```
READ: user U follows {A, B, C, ...}
  getFeed(U)
    -> lookup followee list for U            (graph store)
    -> fetch recent posts for A, B, C, ...    (fan-out reads, parallel)
    -> merge by timestamp, paginate, return

POST: just write the post once. O(1) write. FAST & CHEAP.
```

- **Pro:** writes are trivial — one row, no matter how many followers. No celebrity problem.
- **Con: reads become expensive** — every feed view fans out into N parallel reads (one per followee) plus a merge step. A user who follows 2,000 accounts triggers 2,000 lookups on every single feed refresh. Given reads outnumber writes 40:1+, this is the wrong tradeoff for most users.

The hybrid (what real systems do — Twitter's actual design):

```
Normal user posts        -> fan-out ON WRITE (push into followers' Redis feed lists)
Celebrity/high-fanout     -> DO NOT fan out; just store the post
  user posts

READ getFeed(U):
  1. Read U's precomputed feed from Redis (covers all "normal" followees U has).
  2. Read U's followed-celebrities list (small; a few dozen at most, since celebrity
     accounts are a tiny slice of all accounts) and PULL their recent posts directly.
  3. Merge step 1 + step 2 by timestamp -> return page.
```

- Threshold: "celebrity" = follower count above some cutoff (e.g., 1M). Enforced in the `follow` write path — when U follows a celebrity, tag that edge so fan-out workers skip it.
- **Why this wins:** the *vast majority* of accounts are normal, so push keeps reads O(1) for the common case. The rare celebrity accounts are pulled at read time by the *much smaller* set of users

who follow them — and pulling from a handful of celebrities (not thousands of regular followees) is cheap. **Say this hybrid explicitly — it's the answer interviewers are listening for.**

4. API style — REST vs gRPC vs GraphQL

Choice: GraphQL for the client-facing feed API; gRPC between internal services.

```
query HomeFeed($cursor: String) {
  feed(cursor: $cursor) {
    posts {
      id, text, createdAt, media { url }
      author { id, name, avatarUrl }
      likeCount, isLikedByMe
    }
    nextCursor
  }
}
```

Why GraphQL: a single feed screen needs post content + author profile + media URLs + like counts *in one response* — a classic nested, multi-entity fetch. REST would mean either one bloated custom endpoint or several round trips (N+1: fetch posts, then fetch each author). GraphQL lets the client (which varies — mobile vs. web, different fields needed) shape the query itself without the backend hand-rolling a new endpoint per client. **Why not plain REST:** we'd end up building a bespoke "feed with embeds" endpoint anyway — that's GraphQL by another name, minus the flexibility. **Why gRPC internally:** the feed service calling the fan-out workers, the post service, or the graph service is service-to-service, high volume, and benefits from typed contracts + low overhead — REST/GraphQL's flexibility isn't needed there.

5. Idempotency — needed?

Yes, on post creation. - A flaky network can cause a client to retry `createPost` after a timeout, risking a duplicate post. Client sends a **client-generated** `Idempotency-Key` (e.g., a UUID made at compose time); the post service checks a short-lived key→postId cache — if seen, return the existing post instead of creating a new one. - **Fan-out itself must also be idempotent**: a fan-out worker can crash and be retried, or a queue can redeliver a message. Prepending the same `postId` into a follower's feed list twice must be a no-op — either dedupe by checking membership before insert, or use a data structure (e.g., a sorted set keyed by postId) where re-inserting the same ID is naturally idempotent.

6. Network protocol

HTTPS/HTTP2 for normal API calls (post, follow, paginated feed fetch — plain request/response, no server push needed for a page load). **WebSocket or long-poll, optional, for live updates** — e.g., a "N new posts" banner at the top of an open feed screen. This is a nice-to-have, not core: the base feed product works fine on polling/refresh alone, so don't over-engineer it unless asked.

7. Database — SQL vs NoSQL

Three different stores for three different access patterns — don't force one DB to do everything.

- **Posts → wide-column / KV store (Cassandra or DynamoDB).** Access pattern is "write once, read by ID or by author, at huge scale, no joins." Cassandra's partition-by-author + clustering-by-time layout is a perfect fit, and it scales writes horizontally — important since posts are the highest-volume write in the system. **Why not SQL:** at this write volume, and with no need for cross-post transactions or joins, a relational DB adds cost (harder to shard) for no benefit.
- **Social graph (follows) → graph DB or adjacency-list tables.** The core query is "who does U follow" / "who follows U" — a graph DB (or a simple `follows(follower_id, followee_id)` table indexed both directions) handles this natively. A dedicated graph DB (Neo4j-style) helps if we ever need multi-hop queries ("friends of friends"); for a feed system, simple adjacency-list tables in a KV/SQL store are usually enough.
- **Precomputed feeds → Redis (list/sorted-set per user).** This is the hot path — read on every feed view. In-memory, sub-millisecond, and naturally expresses "an ordered list of post IDs per user." **Why not the primary post store for this:** we need push updates and O(1) reads-by-user, which is exactly a cache/in-memory structure's job, not a durable system-of-record's job (Redis here is the source of truth for *feed order*, but the posts themselves are durably stored elsewhere and can rebuild the cache if lost).
- **Media (images/video) → object storage (S3) + CDN**, never in a database — large binary blobs belong outside the DB; the DB just stores a URL/reference.

8. Data model (detailed)

```
Table: users
-------------------------------------------
 user_id (PK), name, avatar_url, follower_count, created_at

Table: follows   (adjacency list, indexed both directions)
-------------------------------------------
 follower_id (PK/partition), followee_id (clustering key)
 -- reverse index for fan-out: followee_id -> list of follower_ids
 is_celebrity_edge (BOOL)   -- set if followee_id is above the fan-out threshold

Table: posts   (Cassandra: partition by author_id, cluster by post_id desc)
-------------------------------------------
 post_id (Snowflake ID -- time-sortable, unique w/o coordination)
 author_id, content, media_url, created_at

Snowflake ID layout (64 bits): [ timestamp | worker_id | sequence ]
  -> IDs sort by creation time and are generated with no central counter,
     no cross-node coordination -> perfect for a high-write, distributed system.

Feed cache (Redis, per user):
  key:  feed:{userId}
  val:  sorted set of postId, score = timestamp (or Snowflake ID itself,
        since it's already time-ordered) -> newest-N kept, older trimmed.
```

9. Load balancer — L4 vs L7 vs API Gateway

```
  Client
    |
 [ L7 LB / API Gateway ]  <- TLS termination, auth, rate limiting
    |                         (e.g., cap posts/min per user to fight spam)
    +--> Post Service      (write: create post, checks idempotency key)
    +--> Graph Service     (follow/unfollow, "who follows whom")
    +--> Feed Service      (read: assembles + returns the home timeline)
```

Choice: L7/API Gateway. We route by path to independently-scaled services — post creation, follow/graph mutations, and feed reads have wildly different load profiles (feed reads dominate at 40:1+), so they need separate autoscaling groups, which requires L7's path-based routing. The gateway is also where we enforce per-user rate limits on posting (anti-spam) and authentication — both L7 concerns.

10. Caching + data flow (the write path through the queue)

New posts must trigger fan-out **asynchronously** — a post service shouldn't block the user's "post" button on fanning out to (up to) millions of followers.

```
WRITE (post) path:
  Client -> Post Service: create post
    -> write post to Cassandra (durable, source of truth)
    -> publish { postId, authorId } to Kafka topic "new-posts"
    -> return success to client IMMEDIATELY (fan-out happens off the critical path)

  Kafka "new-posts" topic
    -> [ Fan-out Worker pool ] consumes each event
         if author is a normal user:
           look up follower list -> for each follower, push postId into
           Redis feed:{followerId} (capped/trimmed sorted set)
         if author is a celebrity (is_celebrity_edge):
           SKIP fan-out entirely -- do nothing here, handled at read time

READ (feed) path (cache-aside + hybrid pull):
  getFeed(U)
    -> read Redis feed:{U}                (pushed posts -- fast path)
    -> read U's followed-celebrity list (small) -> pull their latest posts
       from Cassandra directly
    -> merge both by timestamp -> paginate -> return
    -> Redis MISS (e.g., new user, evicted key) -> rebuild by pulling recent
       posts from ALL of U's followees once, then cache the result
```

11. Concurrency

- **Fan-out workers scale horizontally**: Kafka partitions the "new-posts" topic (e.g., by `authorId`), and a consumer group of workers processes partitions in parallel — more workers, more throughput, with no shared mutable state between them.
- **Idempotent feed inserts**: as in Section 5, a worker crash-and-retry, or an at-least-once queue redelivery, must not double-insert a postId into a follower's feed. Using a Redis sorted set keyed by postId makes re-insertion a safe no-op.

- **Concurrent follow/unfollow** during an in-flight fan-out (e.g., X unfollows the author mid-fan-out): acceptable to have a brief inconsistency (X might still get that one post, or might not) — feeds are already eventually consistent, so this is a non-issue, not a bug to chase.

12. Replication — needed?

Yes, throughout the stack. - **Cassandra**: replication factor 3, quorum reads/writes — survives node failure, and read-heavy access (a post is read by many followers) benefits from spreading load across replicas. - **Redis feed cache**: replicated (primary + read replicas, or a clustered mode) — losing the cache shouldn't mean losing feeds, just a slower rebuild from Cassandra. Because feed order is "mostly recent posts," brief replica staleness is invisible to users. - **Kafka**: partitions replicated across brokers so a broker failure doesn't drop in-flight fan-out events.

13. Multi-geo

```
Each user's home data (profile, own posts, own feed cache) lives in a
home region, close to where they mostly use the app. Posts replicate
asynchronously to other regions for followers who read from elsewhere.

Cross-region following edge case:
  User in EU follows a celebrity whose posts are authored/homed in US.
  -> Fan-out (or pull, if celebrity) crosses regions -> expect extra
     latency (cross-region replication lag, typically sub-second to a
     few seconds) before that post shows up in the EU follower's feed.
  -> Acceptable: feeds are already eventually consistent; a global
     product cannot make "instant everywhere" a hard requirement
     without enormous cost.
```

14. Failure handling

```
Kafka backlog builds up (fan-out workers fall behind)
  -> posts are still safely durable in Cassandra + queued in Kafka;
     followers just see the new post a bit LATE. Feed delivery
     degrades to "eventual," not "lost." Scale out workers to catch up.

Redis feed cache miss / eviction
  -> rebuild that one user's feed on demand by pulling their followees'
     recent posts from Cassandra (slower one-time cost, then re-cached).

Celebrity post = hot key
  -> because celebrities are excluded from push fan-out, there's no
     write hot-spot; on the READ side, their posts get heavy CDN/cache
     treatment (recent-posts-by-author is itself cacheable, since many
     followers pull the SAME small set of posts).

Fan-out worker crash mid-batch
  -> Kafka redelivers the event to another worker (at-least-once);
     idempotent inserts (Section 5/11) make the retry safe.

A Cassandra node down
  -> RF=3 + quorum keeps serving; no visible impact.
```

Design principle: **a post must never be lost, but it's fine for it to arrive in a follower's feed a little late** — this is the slack that makes queue backlogs and cache rebuilds non-emergencies instead of outages.

15. Tradeoffs & what to say

> "The core tension is fan-out on write vs. fan-out on read: push gives O(1) fast reads but blows up on celebrity write-amplification; pull gives cheap writes but expensive reads for users who follow many accounts. I'd use a hybrid — push for normal users (the vast majority), pull for celebrities (a tiny minority of accounts, so pulling their posts at read time is cheap even though many people follow them). Posts live in Cassandra for write-scalable storage with Snowflake IDs for time-ordering without central coordination; the social graph is adjacency-list/graph-store; precomputed feeds live in Redis. New posts go through Kafka so fan-out is async and never blocks the post action. GraphQL fits the client feed query (posts + authors + media in one call); everything is eventually consistent, which is the right tradeoff since a feed is not a financial ledger."

Follow-ups an interviewer may probe - How would you add ranking (not just reverse-chron)? (Feed becomes a candidate-generation + scoring step — pull more candidates than needed, score with an ML model, then return top-N; this can layer on top of the same push/pull hybrid for candidate sourcing.) - How do you cap unbounded feed growth in Redis? (Trim each user's cached list to the most recent N post-IDs — e.g., 800 — since nobody scrolls back further; older reads fall back to a DB pull.) - What if a user follows 10,000 accounts (a "power follower")? (Even push has to pull for such users at read time, or accept a slower merge — this is the same skew problem in reverse; some systems cap follow counts or treat heavy-followers specially too.) - How do you delete a post everywhere it's been fanned out to? (Tombstone the postId — feed reads filter out tombstoned IDs rather than trying to scrub millions of Redis lists synchronously.)

---

9. Chat / Messaging (WhatsApp / Slack)

The problem in one line. Let users send 1:1 or group messages that arrive in real time, show who's online, confirm delivery/read, and still work if the recipient is offline.

---

1. Requirements

Functional - 1:1 messaging and group messaging (channels/rooms). - Online/presence status ("online", "last seen 5m ago"). - Delivery receipts (sent → delivered → read). - Message history — scroll back and load older messages. - Push notification when the recipient is offline/backgrounded.

Non-functional - **Real-time**: message shows up on the recipient's screen in < ~200 ms when both are online. - **Ordered** within a conversation — messages shouldn't appear out of sequence. - **Durable** — a message is never silently lost, even if the recipient is offline for days. - **Massive concurrency** — millions of clients holding an open connection at once. - **At-least-once delivery** with dedup — retries must not double-send.

2. Estimation

```
Assume 500M daily active users, 50 messages/user/day.
  messages/day = 500M * 50 = 25B/day
  writes/sec   = 25B / 86400 ~= ~290k/sec (peak 2-3x ~= 700k/sec)
Concurrent open connections: ~500M DAU, ~20% concurrently online
  ~= 100M persistent connections at peak.
  One server can hold ~50k-100k WebSocket connections (event-loop I/O)
  -> ~1,000-2,000 connection/gateway servers just for socket fan-out.
Storage: 25B msgs/day * ~200 bytes (id+sender+body+meta) = ~5 TB/day
  -> ~1.8 PB/year (before replication factor). Needs a horizontally
  scalable store, not a single SQL box.
```

3. API style — REST vs gRPC vs GraphQL vs WebSocket

Choice: WebSocket for the real-time channel, REST for everything ancillary, gRPC internally between servers.

```
WebSocket (client <-> gateway): send message, receive message,
  typing indicator, presence updates, delivery/read receipts.
REST: GET /conversations/{id}/messages?before=<ts>  (history/pagination)
      GET /contacts, POST /conversations (create group), auth, profile.
gRPC: chat-gateway <-> message-service <-> presence-service (internal).
```

Why WebSocket for real-time: it's a single persistent, full-duplex TCP connection — the server can *push* a message to the client the instant it arrives, with no per-message handshake. **Why not REST/HTTP polling:** short-polling means constant "any new messages?" requests (wasteful, and still adds polling-interval latency); long-polling helps but still pays a new-connection cost per cycle and

doesn't do true duplex push. **Why WebSocket over raw HTTP for this piece specifically:** it's the only one of the three that gives push with one long-lived connection. **Why REST for history:** paginated reads are stateless request/response — no benefit from a socket. **Alternatives to WebSocket:** MQTT (lightweight pub/sub, popular for mobile/IoT — small headers, built-in QoS levels, good on flaky networks) and XMPP (the historical open chat protocol, used by early WhatsApp/Google Talk) — both solve the same "persistent push channel" problem; WebSocket is the pragmatic default because it's a first-class browser/HTTP-upgrade primitive with ubiquitous library support.

4. Idempotency — needed?

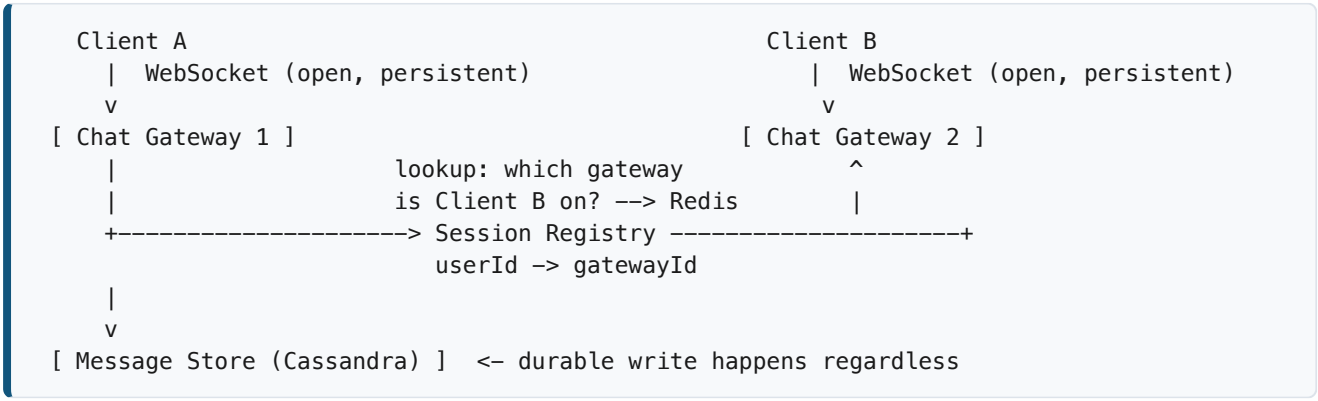
Yes, and it's central here. A flaky connection means the client may resend a message it's unsure was received.

```
Client generates message_id (e.g., UUID) BEFORE sending, client-side.
send(conversation_id, message_id, body) over the socket.
Server: INSERT ... IF NOT EXISTS keyed on (conversation_id, message_id)
  - retry with the SAME message_id -> no-op, server just re-ACKs
  - a genuinely new message -> gets a new message_id -> inserted normally
```

Without this, a network blip + client retry produces two visible copies of the same message. The **client mints the idempotency key**, not the server, because the client is the one that knows "this is a retry of what I already tried to send."

5. Network protocol

WebSocket (TCP) with TLS (wss://), long-poll as a fallback for clients that can't upgrade (old proxies/corporate firewalls). This is the centerpiece of the whole design:



Flow for A → B: A sends over its socket to Gateway 1 → Gateway 1 persists the message → looks up B's `gatewayId` in the **session registry** (Redis) → if B is connected (even on a *different* gateway), forwards the message to Gateway 2 via internal gRPC → Gateway 2 pushes it down B's open socket. **If B is offline** (no entry, or push fails), the message is already durably stored, and a **push notification** (APNs/FCM) is fired instead — B sees the message next time the app opens or via the OS notification.

6. Database — SQL vs NoSQL

Choice: a wide-column store (Cassandra / DynamoDB) for messages; Redis for presence/session; SQL only for small, low-volume metadata (user profiles, group membership).

Why Cassandra for messages: the access pattern is "give me messages for `conversation_id X`, most recent first" — a huge, continuous **write** stream (hundreds of thousands/sec) with **time-ordered** reads and **zero joins**. Cassandra's partition + clustering model maps directly onto that: partition by `conversation_id` (all messages for one chat live together, spread across the cluster by conversation), cluster by `timestamp` (reads come back pre-sorted, newest-first, cheaply). It also scales writes horizontally by just adding nodes — exactly what a "290k writes/sec" workload needs. **Why not SQL as primary:** a single relational instance can't sustain that write volume, and we don't need cross-conversation joins or multi-row ACID transactions — we need fast appends and range scans by key. **Why Redis for presence/session:** presence is small, volatile, read-and- overwritten constantly (heartbeats) — a perfect fit for an in-memory KV store with TTLs; it would be wasteful to burn a durable-store write for every heartbeat.

7. Data model (detailed)

```
Table: messages  (Cassandra, wide-column)
-------------------------------------------------------------
 conversation_id  PARTITION KEY   groups all msgs of one chat
 message_id       CLUSTERING KEY  (time-based UUID -> sortable)
 sender_id
 body
 status                SENT | DELIVERED | READ
 created_at        (redundant w/ message_id, handy for range scans)
-------------------------------------------------------------
 Query: SELECT * FROM messages WHERE conversation_id = ?
        ORDER BY message_id DESC LIMIT 50   -> O(1) partition scan

Table: conversations  (SQL or KV)
 conversation_id PK, type (1:1 | group), member_ids[], created_at

Redis: user_session
 key: "session:{userId}"  -> value: { gatewayId, connectedAt }
 TTL refreshed on heartbeat; missing/expired = user is offline.

Redis: presence
 key: "presence:{userId}" -> "online" | last_seen timestamp, short TTL
```

Why a time-based clustering key (not an autoincrement int): it lets writes from any node land in the right sorted position without a central counter, and "most recent N messages" is a cheap, natural range read.

8. Load balancer — L4 vs L7 vs API Gateway

```
Client
  |
[ L7 LB ]  <- TLS termination, WebSocket upgrade handling,
  |            routes /ws to gateway pool, /api/* to REST services
  +--> Chat Gateway pool (holds long-lived WebSocket connections)
  +--> REST services (history, contacts, auth) — stateless, easy LB
```

Choice: L7, with sticky behavior for the WebSocket path. The initial `GET /ws` (HTTP Upgrade) needs L7 to recognize the Upgrade header and route it to a gateway instance — that's path/protocol-aware routing, an L4 balancer can't see it. **The key nuance for WebSocket LBs:** once the upgrade succeeds, that TCP connection is **pinned to one gateway server for its entire lifetime** — there's no "load balance per request" anymore, it's one long session. So the LB's job is really just picking a gateway *once*, then getting out of the way (L4-style passthrough after the handshake, or L7 for the handshake itself). **Connection draining** matters a lot here: when a gateway is being deployed/retired, the LB must stop sending it *new* connections while letting existing sockets close gracefully (server sends a "reconnect" signal, client reconnects to a fresh gateway) — you can't just kill it, that drops thousands of live conversations at once.

9. Caching + data flow (DB ↔ cache)

```
Session/presence: Redis IS the cache — no separate DB roundtrip on
  every heartbeat; TTL expiry doubles as "user went offline" detection.

Recent messages: cache the last ~50 messages per active conversation
  in Redis (or an in-gateway LRU) so re-opening a chat or a second
  device syncing doesn't hit Cassandra for the common case.
  Cache-aside: read cache -> miss -> read Cassandra -> populate cache.
  Older history (scroll-back) always goes straight to Cassandra —
  not worth caching cold, rarely-read pages.

Write path: message always durably written to Cassandra FIRST,
  then fanned out live to sockets + cache. Never push-then-persist —
  a crash between the two must not lose the message.
```

10. Concurrency

- **Ordering within a conversation:** messages from the same sender are naturally ordered by the client's send sequence; across senders in a group, use the **clustering key (time-based ID)** as the source of truth for display order — don't rely on wall-clock alone across servers (clock skew) — a hybrid logical clock or Cassandra's timeuuid handles ties.
- **Delivery/read-receipt races:** a message can be marked DELIVERED by gateway and READ by the client near-simultaneously; treat status as a **monotonic state machine** (SENT → DELIVERED → READ, never backwards) and use a conditional/last-write-wins update so a late DELIVERED ack can't overwrite an already-READ status.
- **Duplicate sends:** solved by the idempotent `message_id` insert from section 4.
- **Presence flapping:** many rapid connect/disconnects (flaky mobile network) shouldn't spam "online"/"offline" to every contact — debounce presence changes over a short window.

11. Replication — needed?

Yes, for both stores, for different reasons. - **Cassandra, RF=3:** messages are durable, user-generated, and irreplaceable — losing one node must not lose data. Quorum writes/reads balance durability and latency; tunable consistency (e.g., `LOCAL_QUORUM`) keeps writes fast within a region while still replicated. - **Redis (session/presence), replicated:** if a Redis node holding session data dies, we don't want every "is this user online" lookup to go dark — a replica takes over. This data is soft-state (rebuildable from live gateway connections) so replication is about **availability**, not irreplaceable durability — losing the very latest heartbeat is fine.

12. Multi-geo

```
Users connect to the nearest region's gateway pool (GeoDNS / anycast)
-> lowest connection latency for the real-time socket.

Message routing across regions:
  A (region US) sends to B (region EU, currently connected there).
  US gateway persists to its local Cassandra ring (replicated to EU
  asynchronously) AND forwards the live message directly, gateway-to-
  gateway across regions, for immediate delivery — don't wait for
  cross-region DB replication before pushing to B.
Data locality: a conversation's messages are typically replicated to
  all regions its members are in, so scroll-back history is a local
  read wherever a member happens to be.
```

13. Failure handling

```
Gateway server crashes    -> client's socket drops -> client detects
                             (missed heartbeat) -> reconnects to a
                             NEW gateway (via LB) -> re-registers
                             session in Redis -> resumes.
Message sent but recipient's
  gateway unreachable      -> message already durably stored (sec 9)
                             -> retried via a delivery queue; if still
                             unreachable, falls back to push notif.
Redis session entry stale
  (crash w/o clean disconnect) -> TTL on the session key expires ->
                             user treated as offline until they
                             reconnect and refresh it (heartbeat-driven).
Cassandra node down        -> RF=3 + quorum keeps serving; repaired
                             in background (hinted handoff / repair).
Network partition between
  regions                  -> local delivery still works; cross-region
                             messages queue and flush once healed.
```

Design principle: **persist before you push.** The socket layer is best-effort and can drop at any time; the store is the source of truth, so no failure in the real-time path loses data — it just delays delivery.

14. Bottlenecks & scaling

- **Millions of concurrent connections** → gateways are stateless *about message logic* but stateful *about which sockets they hold*; scale the pool horizontally, each server capped by file-descriptor/memory limits (~50k-100k sockets each), fronted by the L7 LB.
- **Session registry lookups** on every message → Redis is in-memory and horizontally shardable by `userId` — keeps this a sub-millisecond hop, not a bottleneck.
- **Write-heavy message store** → Cassandra's partitioning by `conversation_id` spreads load across the ring; a single viral group chat is still bounded by one partition, so extremely large "broadcast" channels may need fan-out sharding tricks (separate hot-partition handling).
- **Group fan-out** (message to a 500-person group) → don't do 500 synchronous pushes inline; enqueue fan-out asynchronously so one slow/offline recipient doesn't block the rest.

15. Tradeoffs & what to say

> "The core insight is that chat needs the server to *push*, so I use WebSockets between client and a stateless-but-connection-holding gateway layer, with a Redis session registry mapping user → gateway so any server can route a message to wherever the recipient is connected. Messages are always persisted to Cassandra — partitioned by conversation, clustered by time — before being pushed live, so a dropped socket never loses data; offline recipients fall back to push notifications. Client-generated message IDs make sends idempotent so retries don't duplicate messages. RF=3 replication and a per-user TTL'd presence entry in Redis keep it available and self-healing when a gateway or node dies."

Follow-ups an interviewer may probe - How do you scale group chats with thousands of members? (Fan-out service, async delivery, maybe a separate "broadcast" path instead of per-recipient fan-out.) - How do read receipts work in a group without a write per member per message? (Store last-read pointer per user per conversation, not a row per message per reader.) - End-to-end encryption? (Server routes ciphertext blobs; key exchange is a separate concern, server never sees plaintext.) - What if a user has multiple devices open? (Session registry maps user → *set* of gateway/device pairs; fan out to all of them.)

---

10. Notification System

The problem in one line. Let internal services tell users things — via push, SMS, and email — reliably, without duplicates, and without a slow third-party provider ever blocking the service that triggered the notification.

---

1. Requirements

Functional - Any internal service can request a notification: `send(userId, event, channel(s), payload)`. - Support **push** (iOS/APNs, Android/FCM), **SMS** (Twilio), and **email** (SES) — often the same logical event fans out to multiple channels. - **User preferences**: per-channel opt-in/opt-out (e.g., "SMS for security alerts only"). - **Templating**: a marketing/product team edits copy without redeploying code. - **Deduplication**: retries or duplicate triggers must not double-notify a user. - **Rate limiting** per user (don't send 50 pushes in a minute). - **Delivery tracking**: sent / delivered / failed / opened, queryable per notification.

Non-functional - **Write-heavy, async** — callers should never block on a slow SMS provider. - **At-least-once delivery** — better to occasionally retry than silently drop a notification. - **High throughput, bursty** — a marketing blast or an incident alert can spike volume 100x. - **Decoupled from producers** — one flaky channel/provider must not slow down the others.

2. Estimation

```
Assume 50M users, avg 5 notifications/user/day (mixed channels).
  total/day  = 250M events
  events/sec = 250M / 86400 ~= ~2,900/sec avg, peak (campaigns) ~30k/sec
Fan-out: one event -> up to 3 channel sends -> ~750M channel-sends/day.
Delivery log: 750M rows/day * ~200 bytes = ~150 GB/day
  -> a wide-column / KV store, not a relational table, for this volume.
Queue throughput at peak: 30k msgs/sec -> partition/shard the topic
  (Kafka: dozens of partitions; SQS: multiple queues or FIFO groups).
```

3. API style — REST vs gRPC vs GraphQL

Choice: REST for the public/producer-facing submit API; gRPC between internal services.

```
POST /api/v1/notifications
  { userId, eventType, channels: ["push","email"], templateId, data }
  -> 202 Accepted { notificationId }
GET  /api/v1/notifications/{id}   -> delivery status per channel
```

Why REST + 202 Accepted: the caller is another backend service firing an event — it wants a simple, fire-and-forget POST, not a chatty protocol. Returning **202** immediately (before delivery) is the key signal that this is asynchronous — the work happens after the response. **Why gRPC internally:** the notification

service, template service, and preference service talk to each other at high QPS with typed schemas — gRPC's low overhead fits. **Why not GraphQL:** no client is picking-and-choosing nested fields; the shapes are fixed and simple.

4. Idempotency — needed?

Yes — this is the crux of the problem. Producers retry on timeout, and the queue itself gives **at-least-once** delivery, so the same event can arrive twice.

```
dedup_key = hash(userId + eventType + eventId)   (eventId from the producer,
                                                    e.g. "order_shipped:order_123")
Before sending on a channel:
  SETNX dedup:{dedup_key}:{channel} "1" EX 24h   (Redis, atomic)
    -> if key already existed: skip send, log as duplicate-suppressed
    -> if new: proceed to send
```

Without this, a producer retry or a worker re-processing a message after a crash would notify the user twice. The dedup key is **per (user, event, channel)** — a user might legitimately get both a push and an email for the same event, but not two pushes for it.

5. Network protocol

HTTPS for the ingest API and internal RPCs. Downstream to providers, whatever they expose: APNs/FCM use their own binary/HTTP2 protocols, Twilio/SES are HTTPS REST. No WebSockets are needed anywhere here — this is fire-and-forget async delivery, not a live channel back to the producer.

6. Database — SQL vs NoSQL

Split by access pattern — two different stores:

```
Templates + UserPreferences  -> SQL (Postgres) or a document store
  - small, low-write-volume, benefit from structure/joins (template
    versions, per-channel opt-in flags) and strong consistency.

DeliveryStatus / DeliveryLog -> wide-column / KV (Cassandra / DynamoDB)
  - 100s of millions of rows/day, append-mostly, queried by
    notificationId or userId+time -> exactly the KV/wide-column sweet
    spot. A relational table would buckle under this write rate and
    gains nothing from joins here.
```

Why this split: preferences/templates are low-volume and relational in nature (few writes, need integrity); delivery logs are the opposite — massive volume, simple key-based access, no joins. Using one store for both would force a bad compromise either way.

7. Data model (detailed)

```
Table: templates
-------------------------------------------------
 template_id   PK
 channel       (push | sms | email)
 version
 body          (with {{placeholders}})
 updated_at

Table: user_preferences
-------------------------------------------------
 user_id       PK
 channel       SK   (push | sms | email)
 opted_in      BOOL
 quiet_hours    (optional, e.g. 22:00-08:00 local)

Table: notifications (metadata, SQL or doc store)
-------------------------------------------------
 notification_id  PK
 user_id
 event_type
 dedup_key
 created_at

Table: delivery_status (KV / wide-column, high write volume)
-------------------------------------------------
 notification_id  PARTITION KEY
 channel          CLUSTERING KEY   (push | sms | email)
 status           (queued|sent|delivered|failed)
 attempt_count
 provider_msg_id
 updated_at
```

8. Load balancer — L4 vs L7 vs API Gateway

```
  Producer services
       |
  [ L7 / API Gateway ]  <- TLS, auth, rate limiting per producer,
       |                    routes POST /notifications
       v
  Notification Service (stateless, validates + enqueues)
       |
  [ Message Queue: Kafka / SQS ]
       |
  +----+----+----------+
  v         v          v
Push      SMS        Email
Workers   Workers    Workers
  |         |          |
APNs/FCM  Twilio      SES
```

Choice: L7/API Gateway — we need auth per producer service, request validation, and rate limiting *before* anything hits the queue (so one noisy producer can't flood it). Pure L4 can't do any of that; it only load-balances connections.

9. Caching + data flow (DB ↔ cache)

The **queue is the backbone** of this system, not the cache — but caching still matters on the read side of the pipeline:

```
Worker processing a message:
  1. look up user_preferences for (userId, channel) -> Redis cache-aside
       HIT (common; prefs change rarely) -> use cached value
       MISS -> read SQL/doc store -> cache with short-ish TTL
     (if opted out, drop here — never call the provider)
  2. look up template by templateId -> Redis cache-aside (templates are
     near-static; long TTL, invalidate on template edit)
  3. render template with payload data
  4. call provider (APNs/FCM/Twilio/SES)
  5. write delivery_status (KV store) -- always, even on failure
```

Caching preferences and templates keeps the hot per-message work off the relational store, which matters when the queue is draining at tens of thousands of messages/sec.

10. Concurrency

- **Many workers per channel** consume the same queue **in parallel** — push, SMS, and email workers scale independently since providers have very different latency/throughput profiles.
- **Ordering** is generally *not* required — two notifications to the same user can be processed out of order without correctness issues (unlike, say, a ledger). This is what makes wide parallelism safe.
- **Per-user rate limiting**: a sliding-window counter in Redis (`INCR user:{id}:count EX 60`) checked before send; if over the cap, delay or drop low-priority notifications.
- **Dedup check-and-set** (section 4) must be atomic to avoid two workers racing on the same event and both sending.

11. Replication — needed?

Yes, at two layers. - **Queue replication**: Kafka partitions each replicated across brokers (e.g., RF=3) so a broker loss doesn't drop in-flight messages; SQS is replicated across AZs by the provider. - **DB replicas**: read replicas for user_preferences/templates serve the cache-miss read load without hitting a single primary; the delivery-log store (Cassandra/Dynamo) is replicated by design (RF=3) and tolerates eventual consistency fine — a status update being a few hundred ms stale doesn't harm anything.

12. Multi-geo

```
Producers write to a queue in their home region.
Regional workers consume locally and call the geographically-nearest
provider endpoint (APNs/FCM/Twilio/SES all have regional entry points)
-> lower latency, and keeps EU user data in EU if required.
User preference/template data replicates to all regions (read-mostly);
delivery status can stay regional since it's rarely read cross-region.
```

Route by the user's home region, not the producer's, so a US service notifying an EU user still sends from the EU worker pool.

13. Failure handling

```
Provider (e.g., Twilio) down/slow -> retry with exponential backoff
    (1s, 2s, 4s, ... capped) -> after N attempts, move message to a
    Dead Letter Queue (DLQ) for manual/automated re-drive later.
Queue backlog builds up        -> autoscale workers; degrade by
    deprioritizing non-critical notification types first.
One channel fails, others succeed -> per-channel delivery_status is
    independent; a failed SMS does NOT block or roll back the email
    that already succeeded for the same event (partial success is OK
    and expected).
Worker crashes mid-processing  -> message is redelivered (at-least-
    once) -> dedup key (section 4) prevents a duplicate user-facing send.
Notification service itself down -> producers' POSTs fail fast (502);
    producers should retry with their own backoff; nothing is lost from
    the queue's perspective since nothing was enqueued yet.
```

Design principle: **the queue absorbs the mismatch** between fast, bursty producers and slow, rate-limited, sometimes-down providers — that decoupling is the whole point of this system.

14. Bottlenecks & scaling

- **Ingest spikes** (mass campaign) → the notification service just enqueues fast; the queue buffers the burst so downstream providers aren't hit with a wall of traffic at once.
- **Provider throughput caps** (Twilio/APNs rate limits) → per-provider worker pool sized to the provider's limit, with client-side rate limiting/backoff to avoid getting throttled.
- **Delivery-log writes** → wide-column store partitioned by `notification_id`; scales horizontally with more nodes as write volume grows.
- **DLQ growth** during a provider outage → alert on DLQ depth; replay once the provider recovers rather than losing the notifications.

15. Tradeoffs & what to say

> "The core idea is decoupling producers from slow, unreliable third-party providers with a message queue — producers get a fast 202 Accepted, and per-channel worker pools drain the queue at whatever rate each provider can handle. At-least-once delivery is unavoidable with queues, so I add a dedup key per (user, event, channel) checked atomically in Redis to guarantee the user isn't double-notified. Retries use exponential backoff, and anything that exhausts retries goes to a DLQ instead of being silently dropped. I split storage: a relational/document store for low-volume templates and preferences, and a wide-column/KV store for the very high-volume delivery log. Ordering isn't required, so I get easy horizontal scaling of workers; rate limiting is per-user, enforced before the provider call."

Follow-ups an interviewer may probe - How do you avoid sending to a user who just opted out mid-flight? (Check preference at send time, not at enqueue time — cache TTL should be short enough to reflect opt-outs quickly.) - How do you prioritize a security alert over a marketing blast? (Separate queues/topics by priority; workers drain high-priority first.) - How do you know delivery actually succeeded, not just "sent"? (Provider webhooks/callbacks update delivery_status asynchronously to `delivered` / `bounced` / `opened`.) - What if the same event should batch multiple updates into one

notification? (Debounce/ aggregate in the notification service before enqueueing, e.g., "3 new comments" digest.)

---

11. Ride-Hailing (Uber / Lyft)

The problem in one line. Match a rider to a nearby available driver, track the trip live on a map, and price/charge the ride — at massive, constantly-moving, write-heavy location scale.

---

1. Requirements

Functional - Rider requests a ride (pickup + destination) → system finds a nearby driver. - Drivers continuously send their live GPS location. - **Matching**: find nearby available drivers and assign one driver to one trip. - **Live tracking**: rider sees driver's car moving on the map in real time. - Fare estimation and final pricing, including **surge pricing** under high demand. - Trip lifecycle: request → match → pickup → ongoing → drop-off → payment.

Non-functional - **Write-heavy on location**: millions of drivers ping every 3-5s → huge sustained write throughput, far more than trip writes. - **Low-latency matching** (< 1-2s to offer a driver) — riders abandon if it's slow. - **Strong consistency on assignment**: a driver must never be double-booked. - **Eventual consistency is fine for location** — a location a few seconds stale is acceptable; a double-charged rider is not. - Regional/city-local by nature — a trip in Chicago never needs a driver in Tokyo.

The centerpiece of this problem is two things working together: 1. Absorbing a firehose of driver location pings without falling over. 2. Answering "which drivers are within N km of this rider, right now?" in milliseconds, then locking exactly one of them to exactly one trip.

2. Estimation

```
Assume 5M active drivers, ping every 4s during a shift.
  location writes/sec = 5M / 4 ~= 1.25M writes/sec (!)
  -> this alone rules out a relational DB for location updates.

Assume 10M ride requests/day.
  requests/sec (avg) = 10M / 86400 ~= 115/sec (peak ~5-10x -> ~1,000/sec)
Trip record: ~1 KB (rider, driver, route, fare, timestamps).
  10M/day * 1KB = 10 GB/day of trip data -> trivial for SQL, archive over time.
Location record: tiny (~50 bytes: driver_id, lat, lng, ts) but ephemeral —
  we only care about the LATEST position, not history of every ping.
```

The gap between **1.25M location writes/sec** and **~1,000 trip writes/sec** is the whole reason this design splits into two very different storage systems.

3. API style — REST vs gRPC vs GraphQL

Choice: REST for rider/driver-facing requests, WebSocket for live streams, gRPC between internal services.

```
POST /api/v1/rides            { pickup, dropoff }   -> { rideId, status }
GET  /api/v1/rides/{id}       -> trip status, fare, driver info
POST /api/v1/drivers/status   { available: true }   -> driver goes online

WS   /stream/driver/{id}      driver -> server: location pings (every few sec)
WS   /stream/rider/{id}       server -> rider: live driver location + trip events

gRPC (internal): Dispatch <-> Location <-> Pricing <-> Trip services
```

Why REST for request/trip CRUD: simple resource semantics, cacheable GETs, easy for mobile clients. **Why not GraphQL:** no over-fetching problem — each screen needs one shape of data, not flexible nested queries. **Why gRPC internally:** Dispatch calling Location calling Pricing happens many times per second at low latency with typed contracts — perfect for gRPC, wrong job for REST/JSON overhead.

4. Idempotency — needed?

Yes, in two critical places. - **Ride request**: a flaky mobile network means the client may retry `POST /rides` after a timeout, even though the first request succeeded. Require an `Idempotency-Key` (e.g., a client-generated UUID per request attempt) — the server checks "have I already created a trip for this key?" before inserting a new one. Without this, a rider could accidentally end up with **two trips and two drivers**. - **Payment**: charging a card must be idempotent for the same reason — a retried payment call must not double-charge. Payment providers (Stripe etc.) support an idempotency key natively; store the trip's payment attempt key and reuse it on retry.

Location pings don't need idempotency — they're "latest wins," not accumulating state.

5. Network protocol

WebSocket for anything continuous, HTTPS for anything request/response.

```
Driver app  --(WS, location every few sec)-->  Location Service
Rider app   <--(WS, driver position + ETA)---  push updates
Ride request, trip history, payment          -> plain HTTPS (REST)
```

Why WebSocket for location/tracking: the server needs to **push** updates to the rider without the rider polling every second (wasteful, laggy). A persistent duplex connection lets both driver→server pings and server→rider updates flow over one connection. **Why not WebSocket for everything:** request/response calls (create ride, fetch trip) don't need a persistent connection — plain HTTPS is simpler, stateless, and load-balances trivially. Fallback to HTTP long-polling if WS is blocked (some corporate networks/older devices).

6. Database — SQL vs NoSQL

Choice: three different stores for three different access patterns.

```
Trips, Users, Payments  -> SQL (Postgres/MySQL), sharded by city/region
Live driver location    -> Redis (in-memory, geospatial index)
Trip history / analytics -> wide-column store (Cassandra/Bigtable/S3+Parquet)
```

Why SQL for trips/payments: these need **ACID transactions** — assigning a driver, decrementing "available," and creating a trip must happen atomically; money movement demands strong consistency and auditability. Joins (rider ↔ trip ↔ payment) are natural here.

Why Redis (in-memory) for location, not SQL: at 1.25M writes/sec of "just update my current lat/lng," a relational DB would buckle under I/O and lock contention for data we don't even need to persist durably. Redis holds data in memory, supports `GEOADD` / `GEOSEARCH` for radius queries in **O(log N)**, and treats location as disposable, overwrite-in-place state — exactly what it is. We accept that a crash loses the last few seconds of positions (acceptable — drivers resend within seconds).

Why a wide-column store for history: completed trips accumulate forever; we rarely query them transactionally, mostly for analytics/support lookups by driver/rider/date. That's write-once, scan-heavy — the wide-column sweet spot. Move data there asynchronously after trip completion; don't burden the live SQL tables.

7. Data model (detailed)

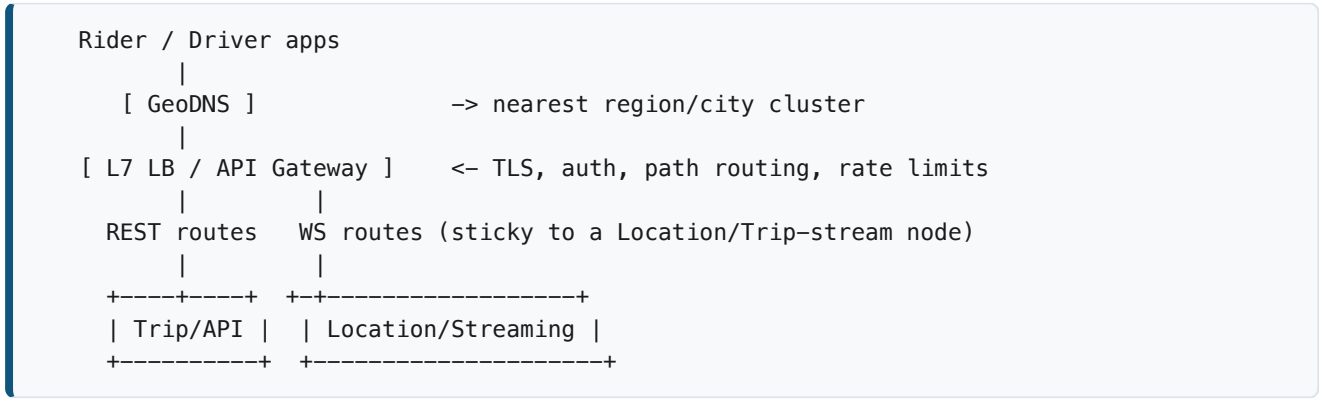
```
Table: drivers          (SQL)
-------------------------------------------------
 driver_id   PK
 status      ENUM(OFFLINE, AVAILABLE, ON_TRIP)
 vehicle_info, rating, current_trip_id (nullable)

Table: riders           (SQL)
-------------------------------------------------
 rider_id    PK
 name, payment_method_id, rating

Table: trips             (SQL)
-------------------------------------------------
 trip_id      PK
 rider_id, driver_id
 status  ENUM(REQUESTED, MATCHED, ONGOING, COMPLETED, CANCELLED)
 pickup_loc, dropoff_loc, fare, surge_multiplier
 requested_at, matched_at, started_at, completed_at
 idempotency_key   UNIQUE   <- dedupes retried ride requests

Table: payments          (SQL)
-------------------------------------------------
 payment_id   PK
 trip_id, amount, status, provider_idempotency_key
```

Geo-index (Redis GEO — the centerpiece)

```
GEOADD drivers:city:sfo  lng lat  driver_id      (on every ping)
GEOSEARCH drivers:city:sfo FROMLONLAT lng lat
          BYRADIUS 3 km ASC COUNT 10             (find nearby)

Under the hood Redis buckets coordinates on a geohash-like grid:

  +--------+--------+--------+
  | cell A | cell B | cell C |   Each cell ~ a geohash prefix.
  +--------+--------+--------+   Rider's cell + 8 neighbors are
  | cell D | RIDER  | cell F |   searched, sorted by distance —
  +--------+--------+--------+   avoids scanning the whole city.
  | cell G | cell H | cell I |
  +--------+--------+--------+

One key per city/region keeps each sorted set small and keeps the
search radius-bounded instead of scanning millions of global points.
```

This is why "geo-indexing" (QuadTree / Geohash / Google S2 cells, or Redis's built-in geohash-based sorted sets) matters: naive "compute distance to every driver" is O(N) per request; a spatial index prunes to nearby cells first, then ranks — turning a citywide scan into a lookup over a few hundred candidates.

8. Load balancer — L4 vs L7 vs API Gateway



Choice: L7/API Gateway. We route `POST /rides` (write, needs auth+rate-limit) differently from `WS /stream/*` (needs **sticky sessions** — a driver's WebSocket connection must stay pinned to the node holding its session state) — that's path-based routing, an L7-only capability. The gateway also handles TLS termination and auth once, instead of every internal service reimplementing it. **Why not pure L4:** it can't distinguish WS streaming traffic from REST traffic or terminate TLS.

9. Caching + data flow (DB ↔ cache)

Redis isn't a cache in front of a DB here — it *is* the system of record for "where is this driver right now." There's no "cache miss → go read SQL" fallback for location, because SQL never stores live coordinates at all.

```
LOCATION path (Redis is source of truth, not a cache):
  Driver app --WS--> Location Service --GEOADD--> Redis (city shard)
  Rider app  --WS<-- Location Service <--GEOSEARCH-- Redis  (poll driver's
                                                        assigned trip)

MATCHING path (read-through nearby drivers, then confirm in SQL):
  Dispatch Service --GEOSEARCH--> Redis: candidate driver_ids
  Dispatch Service --SELECT/UPDATE--> SQL: verify + atomically claim one

TRIP data: normal cache-aside on top of SQL (trip status is read often,
written rarely once matched) -> Redis string cache, short TTL, invalidate on write.
```

10. Concurrency

The core race: two riders request rides near the same driver at the same instant. Both queries hit `GEOSEARCH` and both see that driver as "available" — if both then try to assign him, one rider gets a car that's actually taken.

```
BAD (race):
  T1: read driver.status == AVAILABLE
  T2: read driver.status == AVAILABLE      <- both see AVAILABLE
  T1: set driver.status = ON_TRIP, create trip A
  T2: set driver.status = ON_TRIP, create trip B   <- driver double-booked!

FIX — atomic conditional assignment (compare-and-swap):
  UPDATE drivers
    SET status = 'ON_TRIP', current_trip_id = :tripId
  WHERE driver_id = :id AND status = 'AVAILABLE';
  -- exactly one of T1/T2's UPDATE affects 1 row; the loser gets 0 rows
  -- affected -> dispatch immediately retries with the next candidate.
```

This is a **single-row conditional update** (or a Redis `SETNX` /Lua script if assignment state also lives in Redis) — no distributed lock manager needed, because the "critical section" is exactly one row. Dispatch treats "0 rows updated" as a normal signal to try the next nearby driver, not an error.

Surge pricing is computed from a rolling count of (open requests ÷ available drivers) per city cell, recalculated on a short interval (e.g., every 30s) — an eventually-consistent aggregate is fine; it doesn't need per-request locking.

11. Replication — needed?

Yes, but the story differs per store. - **SQL (trips/payments)**: primary + read replicas per city-region shard; failover via standard leader election. Replica lag is mostly harmless for reads like trip history, but **the assignment write must go to the primary** (no eventual consistency for "is this driver still available"). - **Redis (location)**: replicate for availability (a replica can take over on primary failure), but data is **ephemeral by design** — losing a few seconds of positions on failover is fine because every driver resends within seconds. No need for durable persistence (AOF/RDB) tuned for zero-loss; tune for fast failover instead. - **Wide-column history store**: replicated by design (Cassandra/Bigtable RF=3), since it's write-once and read-rarely, staleness is a non-issue.

12. Multi-geo

```
Cities/regions are natural, near-perfect shard boundaries:
  - a trip's rider, driver, and route all live in one city.
  - matching NEVER needs to search across cities.

  [ SFO cluster ]   [ NYC cluster ]   [ LON cluster ]
    Redis geo         Redis geo         Redis geo
    SQL shard         SQL shard         SQL shard

A rider's app connects to their home city's cluster (or nearest region
for a traveling rider); trips are created and matched entirely within
that region. Cross-region only needed for global rider profile/billing
rollups, which can replicate asynchronously to a central store.
```

This geo-partitioning is a gift for this problem: unlike a URL shortener (must be globally reachable for any key), a ride request is *inherently* local, so sharding by city keeps both the geo-index and the SQL shard small and fast.

13. Failure handling

```
Driver app disconnects        -> WS drops -> location goes stale.
                                 TTL/expire driver's Redis geo entry after
                                 N seconds of no ping -> mark OFFLINE, drop
                                 from nearby-search results (never dispatch
                                 to a driver who might be gone).
Dispatch offer times out/     -> candidate driver doesn't respond in time ->
  driver declines                dispatch immediately retries next-nearest
                                 candidate from the same GEOSEARCH result.
No drivers found nearby       -> widen search radius progressively, or
                                 return "no drivers available" gracefully.
Payment fails after trip ends -> trip still marks COMPLETED; payment retried
                                 async (idempotent) with backoff; flag account
                                 if it keeps failing — never block the rider
                                 from exiting the app over a payment retry.
Redis node/shard down         -> replica promotes; brief gap in matching for
                                 that city is far better than losing durability
                                 guarantees on trips/payments (different store).
Region down                   -> that city's rides are impacted (matching is
                                 local anyway); other regions unaffected.
```

Design principle: **a stale or missing location fails safe** — worst case is a slower match, never a wrong charge or a lost trip record.

14. Bottlenecks & scaling

- **Location write volume** → the single biggest scale axis; solved by keeping it in-memory (Redis), sharding by city, and treating each ping as an overwrite (no accumulating history in the hot store).
- **Matching latency** → geo-index bounds the search to nearby cells instead of scanning all drivers; keep per-city Redis key sets small by sharding geographically.
- **Hot cities at rush hour** → scale Redis/dispatch instances per city independently; a surge in Manhattan shouldn't need capacity in every other city.

- **Trip/payment writes** → modest volume (~1K/sec) — standard SQL sharding by city/region handles it comfortably; archive completed trips out of hot tables.
- **WebSocket fan-out** → many riders watching one driver's position (rare, but possible) — use a pub/sub layer (Redis Pub/Sub or Kafka) so one location update fans out to N subscribers without N separate DB reads.

15. Tradeoffs & what to say

> "The core tension is a firehose of ephemeral location writes versus a small number of transactional trip/payment writes, so I split storage by access pattern: Redis with geospatial indexing (geohash-based sorted sets, conceptually a QuadTree/S2 grid) as the live source of truth for driver location, SQL for trips/payments where ACID matters, and a wide-column store for cold trip history. Matching does a radius search in Redis for candidates, then locks exactly one with a single conditional UPDATE on the driver row — a compare-and-swap, not a distributed lock — so two riders can never claim the same driver. WebSocket carries the continuous streams (location, live tracking); REST/HTTPS handles discrete requests. Cities are natural shards, so both the geo-index and SQL scale by region with no cross-shard matching. Idempotency keys protect ride requests and payments from network retries creating duplicates or double charges."

Follow-ups an interviewer may probe - How do you pick the geo-indexing granularity? (Cell size tradeoff: too coarse -> too many candidates to rank; too fine -> too many neighbor cells to union.) - How do ETAs get computed? (Routing engine, often a separate service using road- graph data, not something the geo-index itself provides.) - What if a driver's GPS is noisy/jumps? (Smooth with a Kalman filter or simply ignore location jumps beyond a plausible speed threshold.) - How would you handle carpooling (multiple riders, one driver)? (Trip model needs a one-to-many relation; matching becomes a mini bin-packing/routing problem.)

---

12. Nearby Places / Proximity Service (Yelp)

The problem in one line. Given a user's location and a radius, quickly return the businesses/places within that radius — a "what's near me" query at map-browsing scale.

---

1. Requirements

Functional - `search(lat, lng, radius, category?)` → list of nearby places, sorted by distance. - `getPlace(placeId)` → full place details (hours, photos, reviews summary). - Business owners can add/update a place (name, location, category, hours) — rare, low volume.

Non-functional - **Read-heavy**: searches vastly outnumber place edits (~10,000:1). Places change rarely (a restaurant doesn't move; hours/menus update occasionally). - **Low latency** on search (< 100–200 ms) — it's an interactive map/list UI. - **Approximate freshness is fine** — a new restaurant appearing with a few minutes of delay is acceptable; this is *not* a strongly-consistent system. - Must scale to **millions of places** and **tens of thousands of QPS** of searches, concentrated in dense urban areas (hot spots).

2. Estimation

```
Assume 50M places worldwide; 20,000 search QPS at peak (city lunch rush).
  Place edits: ~10k/day ~= 0.1 writes/sec -> trivially small vs. reads.
Storage: 50M places * ~1 KB (name, coords, category, metadata) = ~50 GB.
  Tiny for a DB; the CHALLENGE is not storage, it's QUERY SHAPE:
  "find all rows within X km of (lat,lng)" is not a simple key lookup or
  range scan on a normal B-tree index -- lat and lng are two independent
  dimensions, and a B-tree only helps with one at a time.
```

This is the crux of the problem: **the index structure**, not the data volume.

3. API style — REST vs gRPC vs GraphQL

Choice: REST for the public search API; **GraphQL optional** for place-detail pages that aggregate several sub-resources (photos, reviews, hours) in one round trip.

```
GET /api/v1/search?lat=37.77&lng=-122.41&radius=2km&category=coffee
    -> [ { id, name, lat, lng, category, distance_km }, ... ]
GET /api/v1/places/{placeId}   -> full place details
```

Why REST: search takes a handful of query params and returns a list — a plain resource GET, cacheable by URL. **Why GraphQL as an option:** a place-detail screen often needs place info + reviews + photos + hours in one call; GraphQL avoids four separate REST round trips. **Why not gRPC externally:** public web/mobile clients want simple HTTP+JSON; gRPC is more useful *internally* between the search service and the geo-index service.

4. Idempotency — needed?

Reads: yes, trivially — searches are safe/idempotent by nature (no side effects). - `getPlace` and `search` can be retried freely; no dedup logic needed. - `createPlace` / `updatePlace` (rare, owner-driven) should still use an **idempotency key** or a deterministic `place_id` (e.g., client-generated UUID) so a retried "add my business" submission doesn't create a duplicate listing. **Say this explicitly** even though writes are a minor part of the system — interviewers check you didn't skip it.

5. Network protocol

HTTPS (HTTP/1.1 or HTTP/2) over TLS. Search is classic request/response; HTTP/2 multiplexing helps map UIs that fire several tile/category queries as the user pans/zooms. No need for WebSockets — there's no server-push requirement (unless you add "live wait time" updates, which would be a separate concern).

6. Database — SQL vs NoSQL vs geo-index

Choice: SQL or a document store as the source of truth for place records, PLUS a dedicated geospatial index (Elasticsearch geo queries, or an in-memory quadtree/geohash service) for the "nearby" query itself. Two stores, two jobs:

- **Source of truth (SQL/Mongo):** owns the canonical place row — name, category, hours, owner edits. Needs ACID-ish guarantees for edits, joins with reviews/photos, easy to back up.
- **Geo index (Elasticsearch `geo_point` / `geo_distance` query, or a custom quadtree service):** optimized purely for "which IDs are near (lat,lng)". Rebuilt/kept in sync from the source of truth; can be *stale by seconds* without anyone noticing, because places don't move.

Why not just SQL with lat/lng columns: a B-tree index on `lat` or `lng` alone can't answer a 2D radius query efficiently — filtering by `lat` still leaves you scanning a long, thin strip of the world and then filtering by `lng` in memory. You need a structure built for 2D locality. That's the whole point of the next section.

7. THE CENTERPIECE — geospatial indexing

The core problem: "give me everything within X km of (lat,lng)" needs an index where **things that are near each other in the real world are also near each other in the index.**

Naive approach — scan and filter:

```
for each place: compute haversine_distance(place, query_point)
                keep if distance <= radius
O(N) per query -- fine for 10K places, hopeless for 50M.
```

Approach A — Geohash (encode lat/lng into one base32 string; used by Elasticsearch, Redis `GEOADD`):

```
Geohash interleaves bits of lat and lng into a string. Each extra
character narrows the box by ~5x. Places in the same box share a prefix.

  "9q8yy" covers a ~1.2km x 0.6km box in San Francisco
  "9q8yyk" (6 chars) narrows that box further

  Grid (each cell = one geohash prefix):
  +--------+--------+--------+
  | 9q8yv  | 9q8yy  | 9q8yz  |
  |        |  *You  |        |
  +--------+--------+--------+
  | 9q8yt  | 9q8yw  | 9q8yx  |
  +--------+--------+--------+

Query: compute the geohash prefix for (lat,lng), then look up places
sharing that prefix (and its 8 neighbor cells — see below) via a plain
index/range scan on the geohash string column.
```

Approach B — Quadtree (recursively subdivide space; a node splits into 4 quadrants once it holds too many places):

```
      world
    /   |   |   \
   NW    NE   SW   SE          <- split once a cell exceeds, say,
  /|\                              100 places (dense = Manhattan,
 ... (further split, dense area)  sparse = rural = shallow tree)

Dense downtown core -> deep subdivision (small cells, few places each)
Sparse suburbs      -> shallow subdivision (big cells)
```

Quadtrees **adapt to density** — this is their main edge over a fixed geohash grid, which uses uniform cell sizes regardless of how crowded an area is.

Approach C — Google S2 / Uber H3: hierarchical cells on a sphere (avoids geohash's distortion near poles and the "flat grid on a round earth" problem); used by real systems at Google/Uber scale. Worth naming as the production-grade answer, but geohash/quadtree are enough to reason through in an interview.

Recommendation: geohash, backed by Elasticsearch's native geo_point/geo_distance queries (or Redis `GEOADD` / `GEORADIUS` for a simpler in-memory version). It's simple to implement (just a string prefix), maps cleanly onto an existing index (B-tree on a string column), and Elasticsearch already solves the boundary problem for you. Quadtree is the better answer if you're building the index yourself in-memory and want density-adaptive cells; mention S2 as "what Google/Uber actually use" to show depth.

The boundary problem (must address this):

```
You *          |  cell boundary
                |
A place just across the line, closer to you than most
places in your own cell, would be MISSED if you only
query your own geohash cell.

Fix: query your cell + all 8 neighboring cells (a 3x3
block), collect candidates, then filter/sort by true
haversine distance and cut at the requested radius.
```

8. Load balancer — L4 vs L7 vs API Gateway

```
  Client (map app)
      |
  [ GeoDNS ]  -> nearest region
      |
  [ L7 LB / API Gateway ]  <- TLS, path routing, rate limiting
      |
      +--> Search Service  (reads geo-index, read-heavy, autoscaled)
      +--> Place Service   (CRUD on source-of-truth DB, low volume)
```

Choice: L7/API Gateway, routing `GET /search` and `GET /places/{id}` to a horizontally autoscaled, stateless **search service**, while `POST/PUT /places` goes to a small **place service**. L7 lets us scale the hot read path independently of the cold write path, add per-endpoint rate limits, and terminate TLS at the edge.

9. Caching + data flow (DB ↔ cache)

Search is extremely read-heavy and results for a given area/category barely change minute to minute → **cache aggressively, keyed by rounded location + radius + category.**

```
SEARCH read path (cache-aside):
  GET /search?lat=37.77&lng=-122.41&radius=2km&category=coffee
    -> round (lat,lng) to a geohash cell (dedups near-identical queries)
    -> key = "search:9q8yy:2km:coffee"
    -> check Redis
         HIT  -> return cached place-ID list (+ CDN edge cache for popular cells)
         MISS -> query geo-index (Elasticsearch) -> cache with short TTL
                 (minutes, not hours -- new places should surface reasonably soon)
PLACE DETAILS:
  getPlace(id) -> cache-aside on place_id, longer TTL (hours) -- details
  change rarely; a photo/hours edit can just invalidate that one key.
CDN can cache static place data (photos, descriptions) at the edge.
```

Rounding coordinates to a geohash cell is what makes caching effective here — without it, every user's slightly different GPS reading would be a cache miss.

10. Concurrency

- Reads dominate; searches don't contend with each other or with writes in any meaningful way.

- **Place edit vs. concurrent search:** a search might return a place mid-update (e.g., hours just changed) — acceptable staleness, no locking needed.
- **Geo-index rebuild vs. live search:** rebuild into a new index/segment and swap atomically (Elasticsearch does this via segment merges; a custom quadtree service can build the new tree in the background and swap a pointer) — never let readers see a half-built index.

11. Replication — needed?

Yes, heavily, on the read side. - **Read replicas** of the source-of-truth DB for place CRUD reads (owner dashboards, etc.). - **Replicated geo-index shards** (Elasticsearch replicas per shard) so search load spreads across many nodes — this is the real scaling lever, since search QPS dwarfs everything else. - Because places rarely change, **replica lag is harmless** — the same tolerance for staleness as the URL shortener's immutable mappings (problem #1), just for a different reason (low write rate rather than immutability).

12. Multi-geo

```
Places are inherently geo-located: a search for "coffee near me" in
Tokyo never needs data about San Francisco. Partition/replicate the
geo-index BY REGION, and route each search to the region matching the
query's own (lat,lng) -- not necessarily the user's network location.

  [ GeoDNS / query-location routing ]
    -> Americas index shard(s)
    -> EU index shard(s)
    -> APAC index shard(s)

Edge caching of hot-cell search results further reduces cross-region
hops for popular tourist/downtown areas.
```

This is a natural sharding key (unlike, say, a social graph) — location-based partitioning falls out of the problem itself.

13. Failure handling

```
Geo-index (Elasticsearch) node down -> other replicas of that shard
                                       keep serving; rebuild the lost
                                       replica from source-of-truth DB.
Entire geo-index cluster down       -> degrade to a slower fallback
                                       (e.g., cached results only, or
                                       a coarse SQL bounding-box scan)
                                       rather than a hard outage.
Source-of-truth DB down             -> search still works off the
                                       existing index (index is the
                                       hot path, DB is not).
Cache down                          -> fall back to geo-index directly,
                                       higher latency but functional.
Stale index after a place edit      -> acceptable; reconcile via
                                       periodic re-sync or CDC stream
                                       from the source DB into the index.
```

Design principle: **the geo-index is a derived, rebuildable cache of the source of truth** — treat it like one operationally (can always be regenerated; staleness is a known, accepted cost).

14. Bottlenecks & scaling

- **Search QPS in hot cities** → shard the geo-index by region/geohash prefix, add replicas, and cache popular-area queries — the classic power-law hot-spot problem.
- **Dense areas skewing a uniform grid** → this is exactly what a quadtree (or variable-depth geohash) fixes; call this out if using a fixed grid causes imbalanced shard sizes.
- **Index rebuild time** as places grow → incremental updates (CDC from source DB) instead of full rebuilds; only full-rebuild on schema/algorithm changes.
- **Place-detail fan-out** (reviews, photos on a hot restaurant) → cache aggressively, CDN for images.

15. Tradeoffs & what to say

> "The core challenge isn't data volume, it's query shape: 2D radius search doesn't fit a normal B-tree. I'd encode locations with geohash and let Elasticsearch's geo_distance query (or Redis GEORADIUS) do the cell lookup, querying the center cell plus its 8 neighbors to avoid the boundary problem — or use a quadtree if I want density-adaptive cells and I'm building the index myself. Source of truth stays in SQL/Mongo; the geo-index is a derived, rebuildable, replicated structure, so staleness after a place edit is fine. Read-heavy and hot-spot-prone, so I cache popular-area searches keyed by rounded geohash cell, shard the index by region, and route each query to the region matching the query's own coordinates — not the user's network location."

Follow-ups an interviewer may probe - How do you rank results within the radius? (Distance first, then relevance/rating — a secondary sort, not part of the geo-index itself.) - What about a moving user (driving) hitting search repeatedly? (Debounce client-side; cache by rounded cell absorbs most repeat queries.) - How to support arbitrary polygon search (not just radius)? (Elasticsearch geo_shape queries, or point-in-polygon filtering on the geohash candidate set.) - Uber/Lyft driver-matching is the same core problem with movement — see how it changes when the "places" are themselves constantly updating their location.
